



Week 10

Strings and Patterns

Algorithms and Data Structures
COMP3506/7505

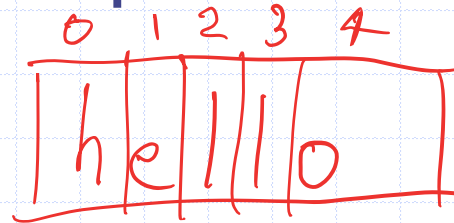
Week 10 – Strings and Patterns

1. Strings & Pattern Matching
2. Tries
3. Suffix Arrays

Strings



◆ **String** – sequence of characters



◆ **Alphabet Σ** – set of possible characters for a family of strings

- $\{0, 1\}$ ←
- $\{A, C, G, T\}$ ←

Strings



◆ Let S be a string of size m

- Substring $S[i..j]$

- Prefix

 - ◆ substring of the type $S[0..i]$ $hell$

- Suffix

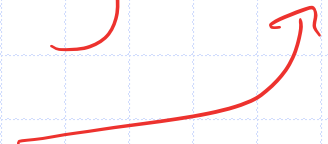
 - ◆ substring of the type $S[i..m-1]$ $ello$
 lo
 o

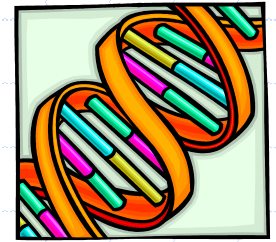
$hello$ $m=5$
 $\begin{array}{c} 0 \qquad 4 \\ \hline e(\\ 12 \end{array}$

Pattern Matching Problem

- ◆ Given strings T (text) and P (pattern)
 - Find a substring of T equal to P

$T = \text{barry malloc is a legend}$
 $P = \text{all}$





Brute-Force Pattern Matching

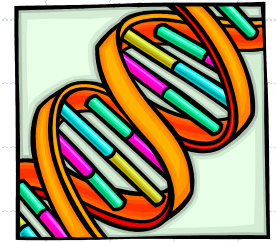
- ◆ Compare the pattern P with the text T for each possible shift of P relative to T

$T =$ A T G C G T C A C $P =$ G C

G C

G C

G C



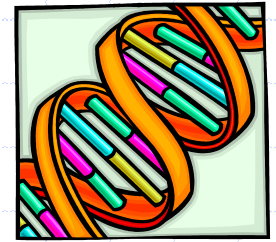
Brute-Force Pattern Matching

Algorithm *BruteForceMatch*(T, P)

Input: text T of size n and pattern P of size m

Output: starting index of a substring of T equal to P
or -1 if no such substring exists

```
for  $i \leftarrow 0$  to  $n - m$  do *  
    { test shift  $i$  of the pattern }  
     $j \leftarrow 0$  *  
    while  $j < m \wedge T[i + j] = P[j]$  do  
         $j \leftarrow j + 1$   
    if  $j = m$  then  
        return  $i$  {match at  $i$ } ✓  
    else  
        break while loop {mismatch}  
return -1 {no match anywhere}
```



Brute-Force Pattern Matching

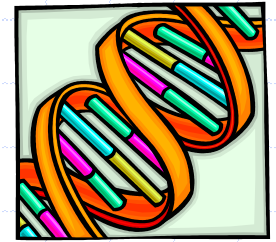
- ◆ Brute-force pattern matching runs in time $O(nm)$
- ◆ Example of worst case:
 - $T = aaa \dots ah$ and $P = aaah$

n^2
 m^2

$T =$

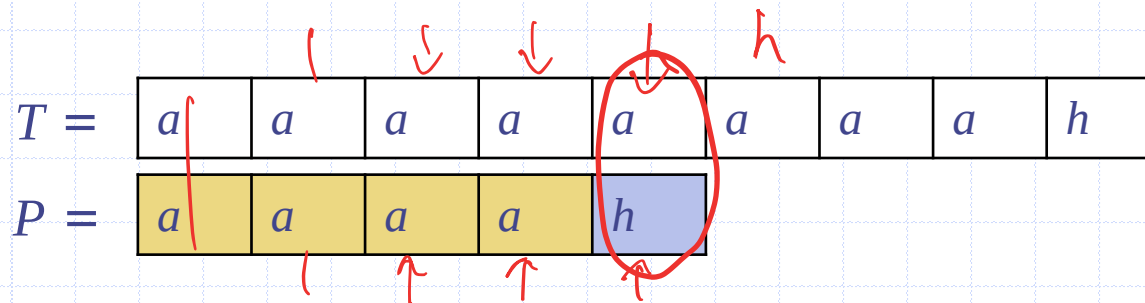
<i>a</i>	<i>a</i>	<i>a</i>	<i>a</i>	<i>a</i>	<i>a</i>	<i>a</i>	<i>a</i>	<i>h</i>
----------	----------	----------	----------	----------	----------	----------	----------	----------

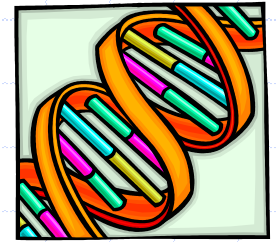
$P =$



Brute-Force Pattern Matching

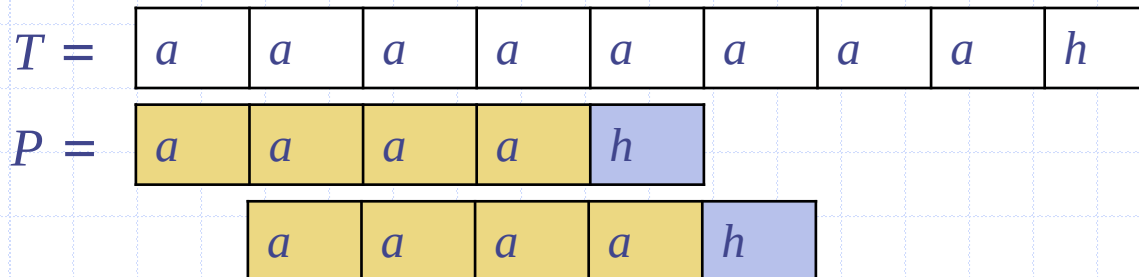
- ◆ Brute-force pattern matching runs in time $O(nm)$
- ◆ Example of worst case:
 - $T = aaa \dots ah$ and $P = aaah$

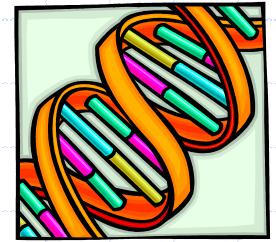




Brute-Force Pattern Matching

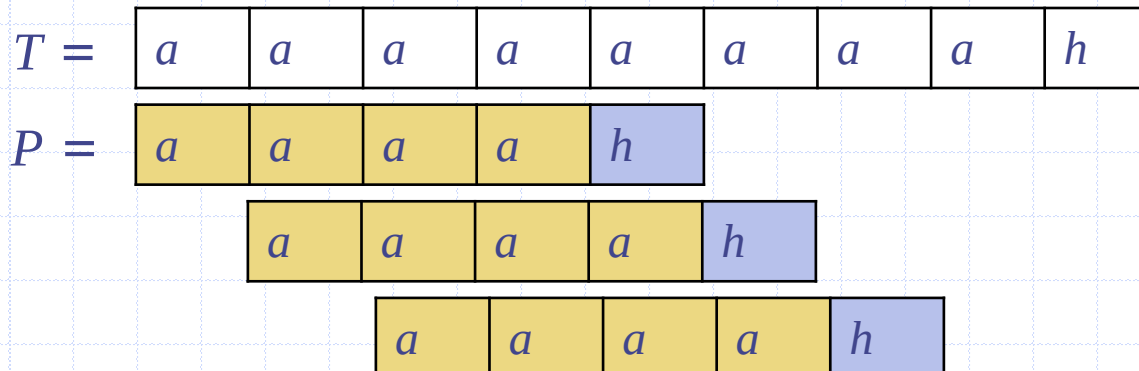
- ◆ Brute-force pattern matching runs in time $O(nm)$
- ◆ Example of worst case:
 - $T = aaa \dots ah$ and $P = aaah$

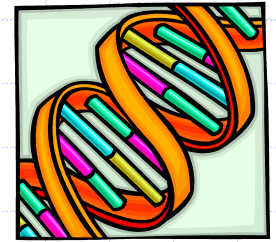




Brute-Force Pattern Matching

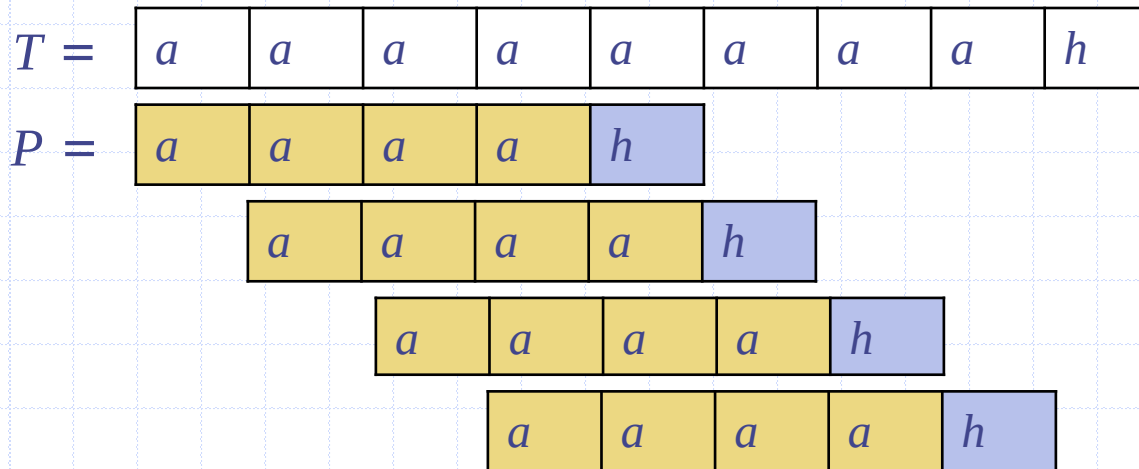
- ◆ Brute-force pattern matching runs in time $O(nm)$
- ◆ Example of worst case:
 - $T = aaa \dots ah$ and $P = aaah$

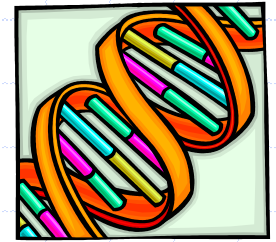




Brute-Force Pattern Matching

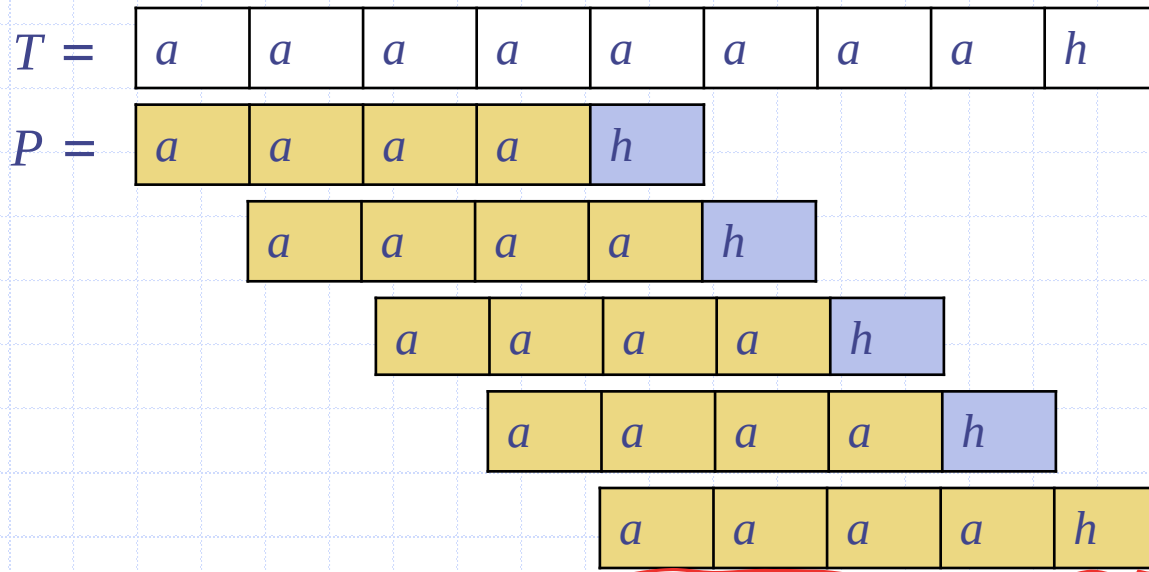
- ◆ Brute-force pattern matching runs in time $O(nm)$
- ◆ Example of worst case:
 - $T = aaa \dots ah$ and $P = aaah$





Brute-Force Pattern Matching

- ◆ Brute-force pattern matching runs in time $O(nm)$
- ◆ Example of worst case:
 - $T = aaa \dots ah$ and $P = aaah$



Can We Do Better?

- ◆ Boyer-Moore pattern matching algorithm
- ◆ Attempts to improve the Brute-Force approach by using two heuristics
 - Looking-glass heuristic
 - Character-jump heuristic

Boyer-Moore: Looking-Glass Heuristic

- ◆ Compare P with a subsequence of T moving backwards

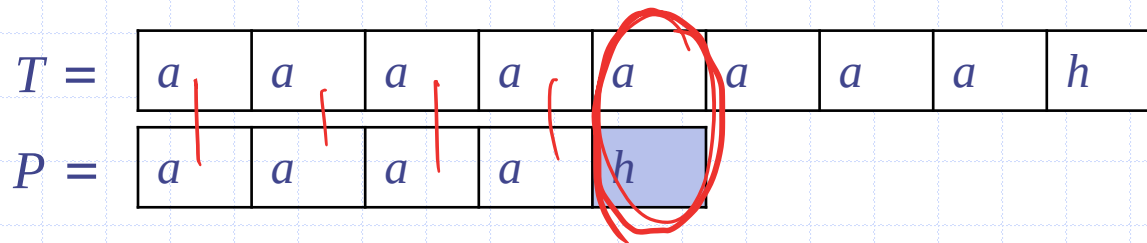
$T =$

a	a	a	a	a	a	a	a	h
-----	-----	-----	-----	-----	-----	-----	-----	-----

$P =$

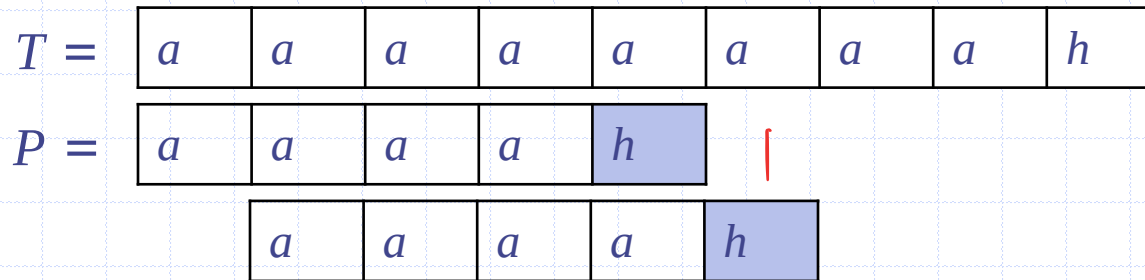
Boyer-Moore: Looking-Glass Heuristic

- ◆ Compare P with a subsequence of T moving backwards



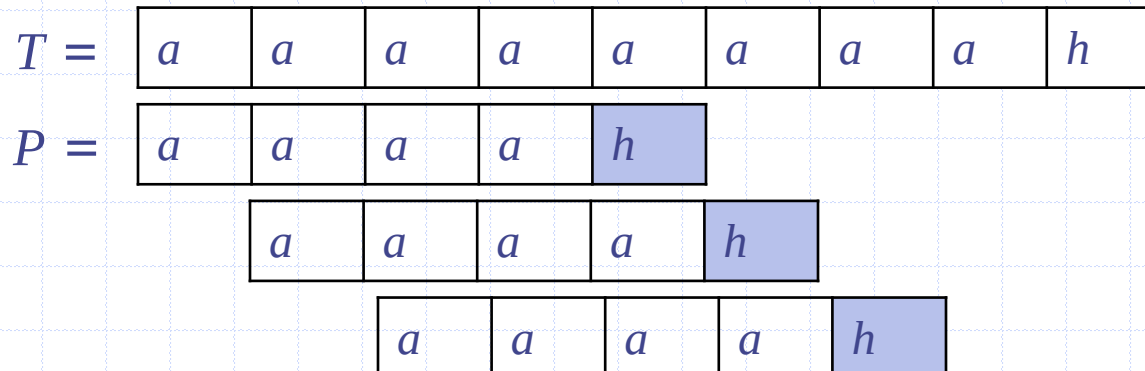
Boyer-Moore: Looking-Glass Heuristic

- ◆ Compare P with a subsequence of T moving backwards



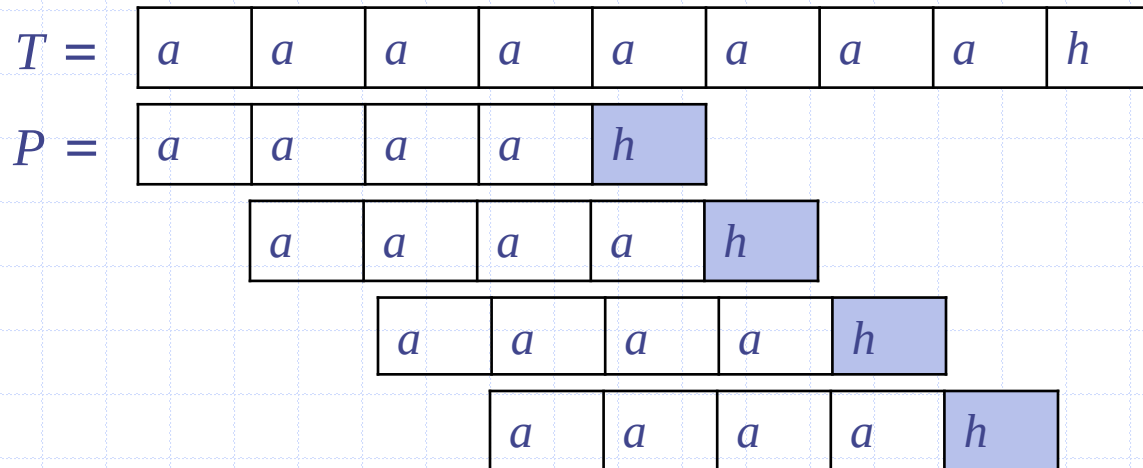
Boyer-Moore: Looking-Glass Heuristic

- ◆ Compare P with a subsequence of T moving backwards



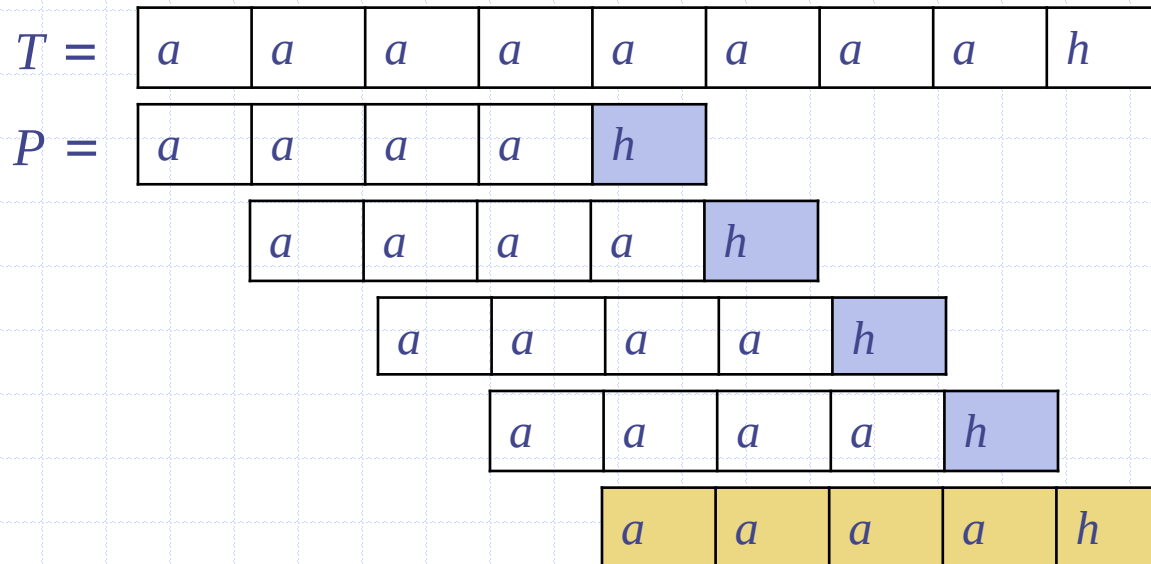
Boyer-Moore: Looking-Glass Heuristic

- ◆ Compare P with a subsequence of T moving backwards



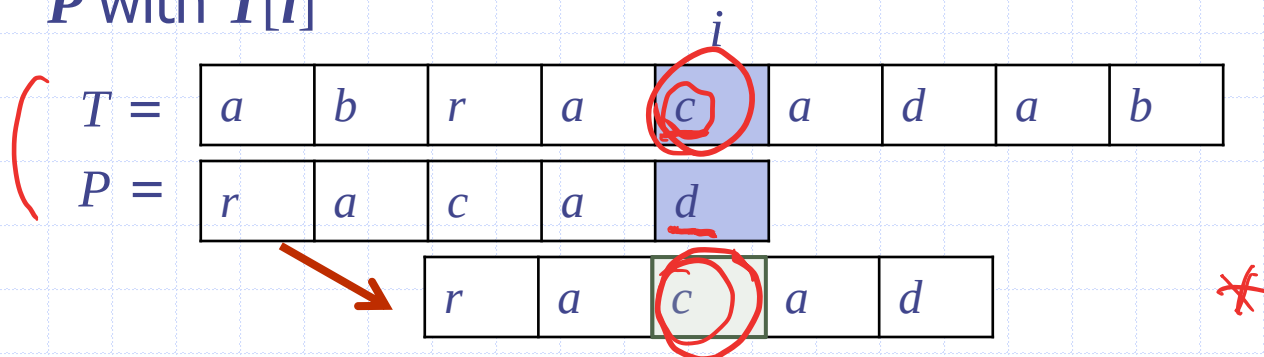
Boyer-Moore: Looking-Glass Heuristic

- ◆ Compare P with a subsequence of T moving backwards

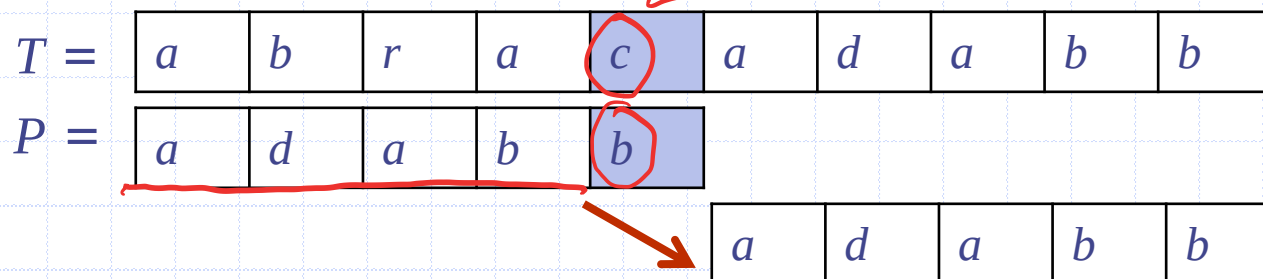


Boyer-Moore: Character-Jump Heuristic

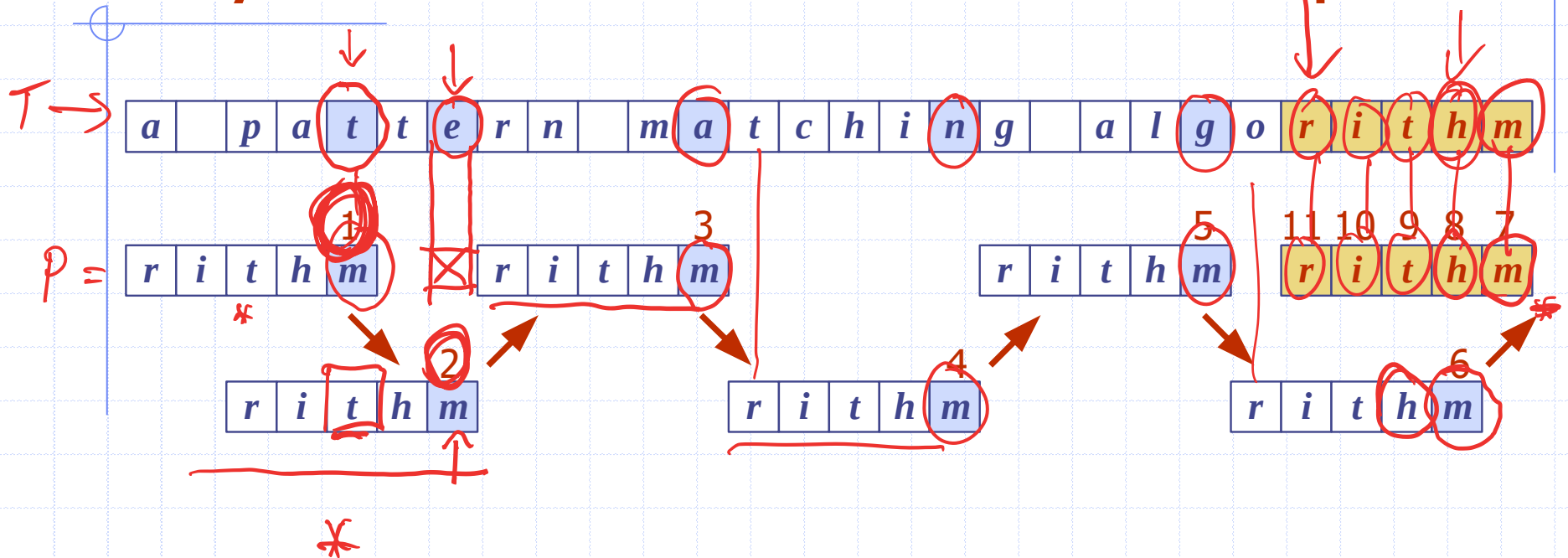
- ◆ When a mismatch occurs at $T[i] = c$
 - if P contains c , shift P to align the last occurrence of c in P with $T[i]$



- Else, shift P to align $P[0]$ with $T[i + 1]$



Boyer-Moore Heuristics: Example



Last-Occurrence Function

- ◆ Boyer-Moore's algorithm preprocesses the pattern P and the alphabet Σ to build the last-occurrence function L mapping Σ to integers, where $L(c)$ is defined as

- largest index i such that $P[i] = c$ or
- -1 if no such index exists

- ◆ Example

- $\Sigma = \{a, b, c, d\}$

- $P = abacab$

0 1 2 3 4 5

~~x~~ abcab

c	a	b	c	d
$L(c)$	4	5	3	-1

$[0, 10, \text{lowercase}]$

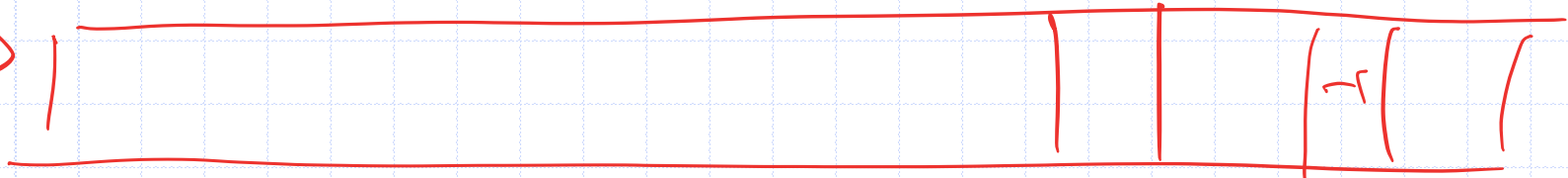
Last-Occurrence Function

◆ Can be represented by an array indexed by the numeric codes of the characters

- computed in $O(m + s)$ time, where m is the size of P and s is the size of Σ

- accessed in $O(1)$ time

32



$\text{Ord}('a') = 10$

Boyer-Moore Algorithm

Algorithm *BoyerMooreMatch*(T, P, Σ)

*** $L \leftarrow \text{lastOccurrenceFunction}(P, \Sigma)$**

$$i \leftarrow m - 1 \quad \{ m \text{ is size of } P \}$$
$$j \leftarrow m - 1$$

repeat

* if $T[i] = P[j]$ then

if $j = 0$ then

```
return i { match at i }
```

else

$$i \leftarrow i - 1$$
$$j \leftarrow j - 1$$

else

{ character-jump }

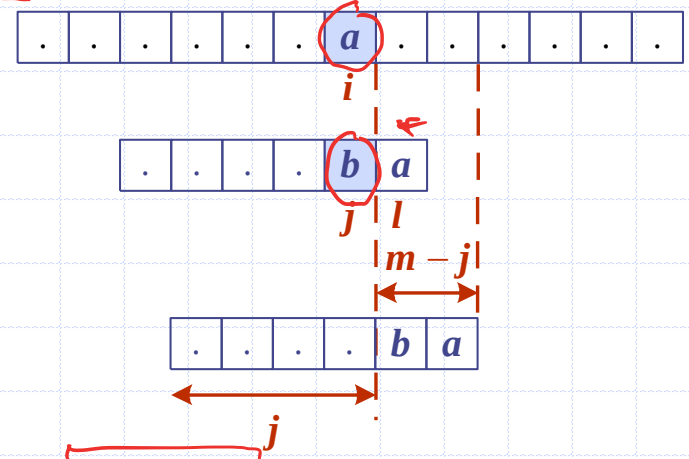
* $l \leftarrow L[T[i]]$

$$\ast \ i \leftarrow i + m - \min(j, 1 + l)$$
$$j \leftarrow m - 1$$

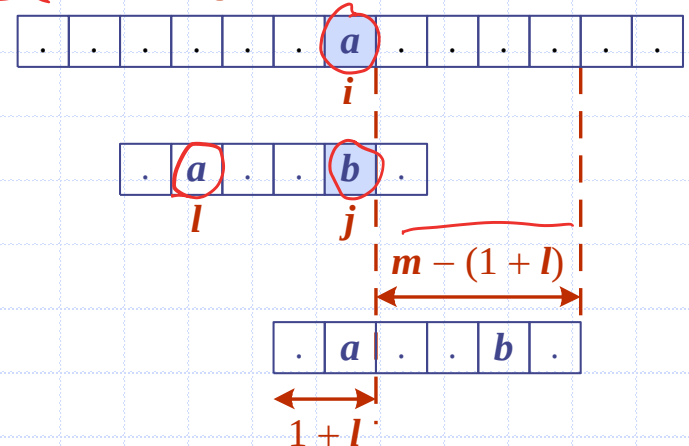
until $i > n - 1$

```
return -1 { no match }
```

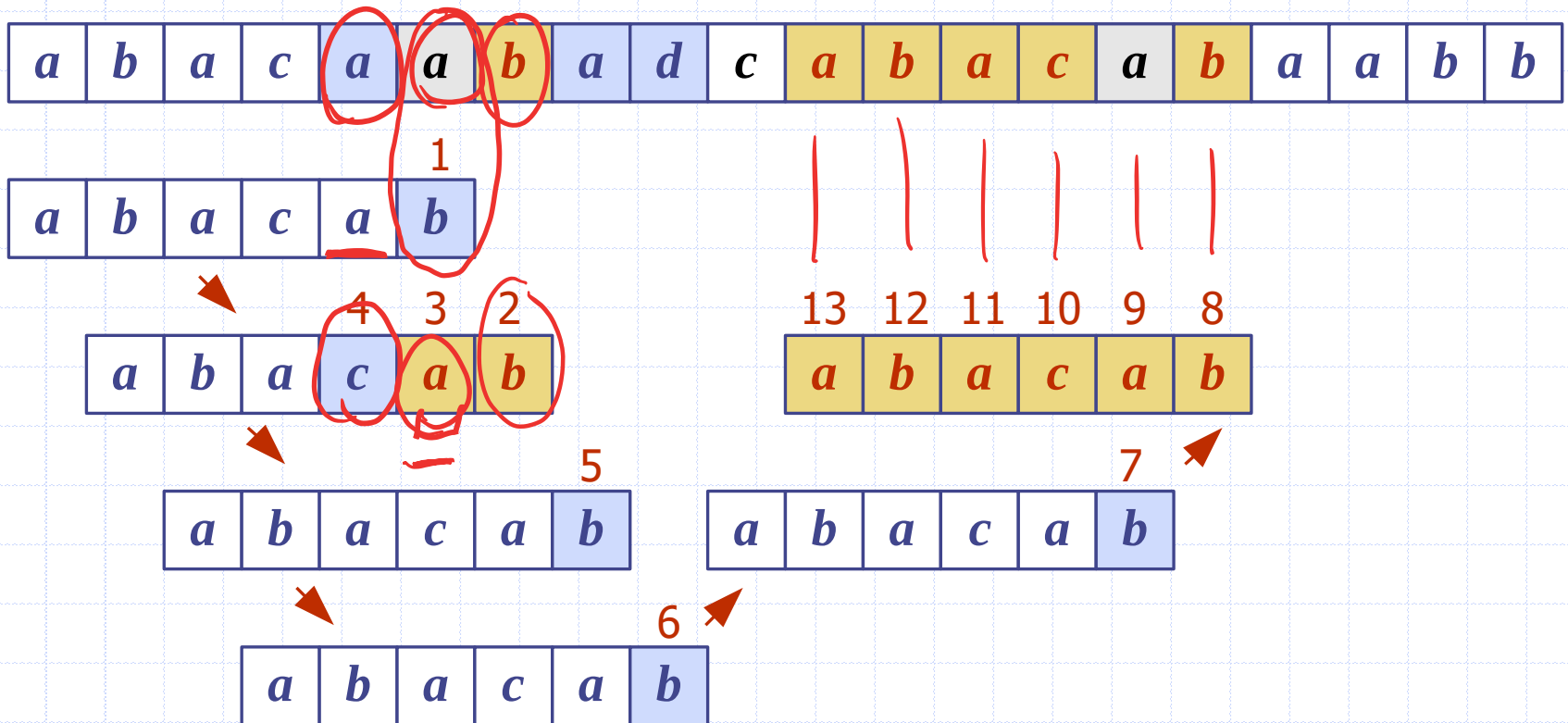
Case 1: $j \leq 1 + l$



Case 2: $1 + l \leq j$

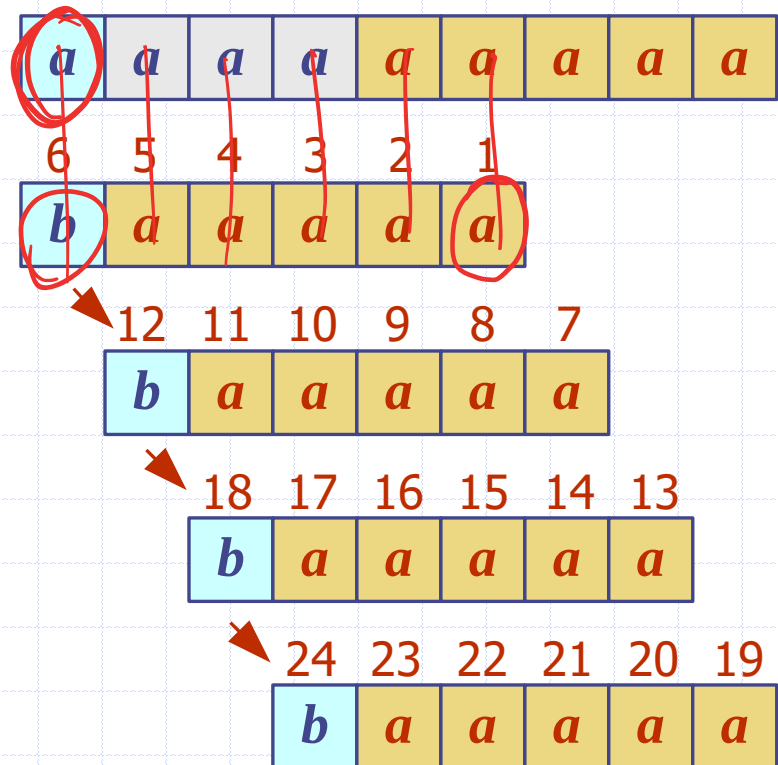


Example



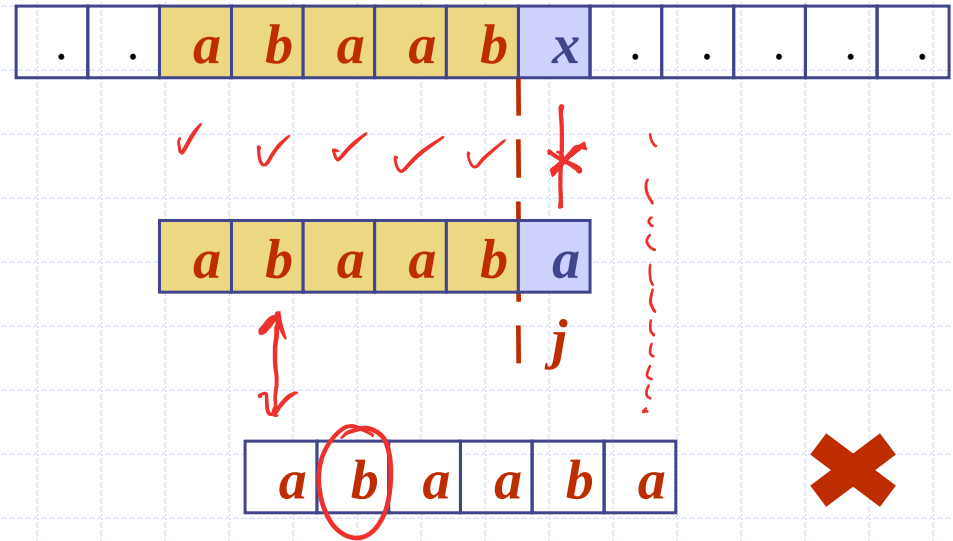
Boyer-Moore's Algorithm Analysis

- ◆ Runs in $O(nm + s)$ time
- ◆ Example of worst case
 - $T = aaa \dots a$
 - $P = baaa$
- ◆ Boyer-Moore's algorithm is significantly faster than the brute-force algorithm



Knuth-Morris-Pratt (KMP) Algorithm

- ◆ Compares the pattern to the text left-to-right
 - but shifts the pattern more intelligently than the brute-force algorithm
- ◆ When a mismatch occurs, what is the most we can shift the pattern so as to avoid redundant comparisons?



Knuth-Morris-Pratt (KMP) Algorithm

$S = \underline{\underline{\text{hello}}}$

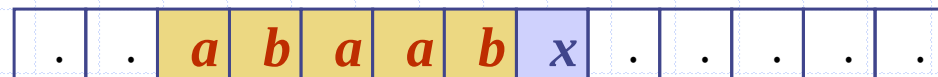
Prefixes
h, he, hel,
ello, llo, lo
Suffixes

- ◆ Compares the pattern to the text **left-to-right**

- but shifts the pattern more intelligently than the brute-force algorithm

- ◆ When a mismatch occurs, what is the **most** we can shift the pattern so as to avoid redundant comparisons?

- ◆ Answer: the largest prefix of $P[0 \dots j-1]$ that is a suffix of $P[1 \dots j-1]$



j

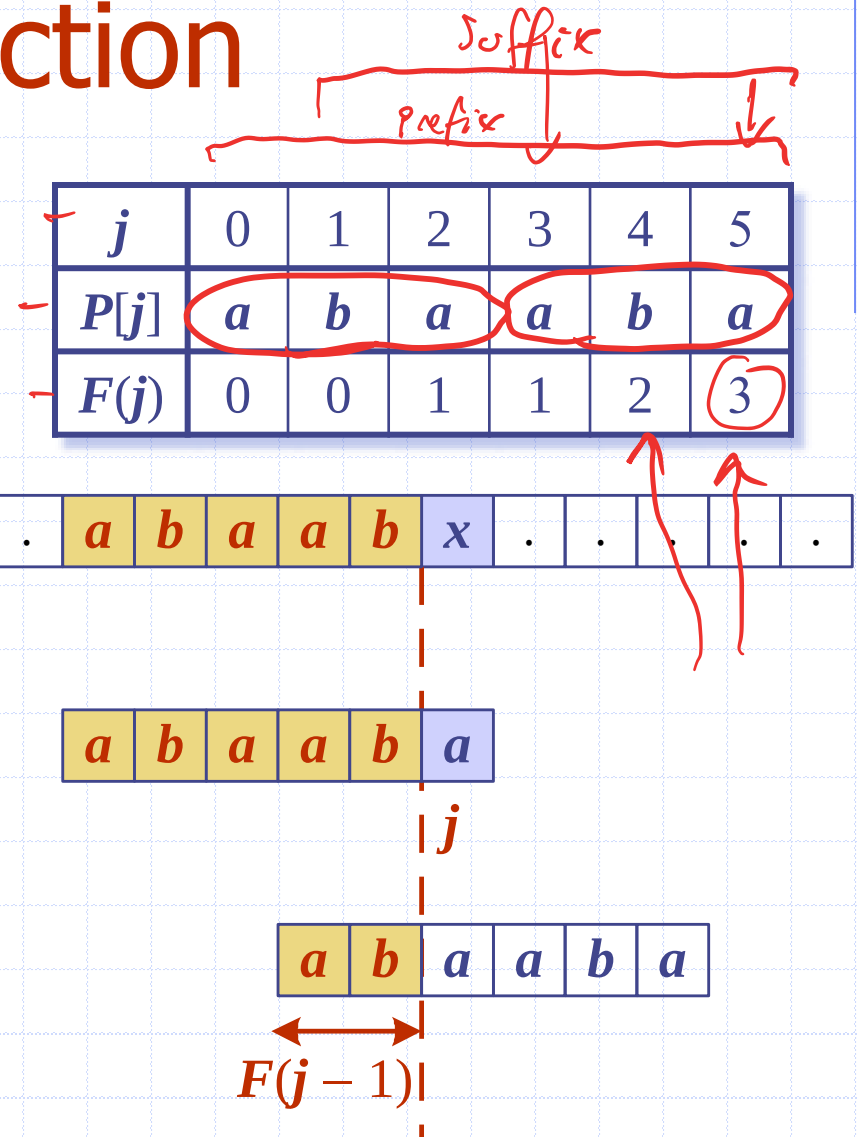


No need to repeat these comparisons

Resume comparing here

KMP Failure Function

- ◆ KMP algorithm preprocesses the pattern
- ◆ Failure function $F(j)$ is defined as the size of the largest prefix of $P[0..j]$ that is also a suffix of $P[1..j]$
- ◆ KMP algorithm modifies the brute-force algorithm so that if a mismatch occurs at $P[j] \neq T[i]$ we set $j \leftarrow F(j-1)$



KMP Algorithm

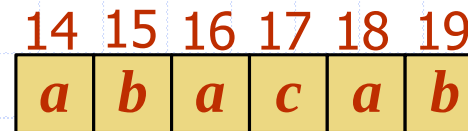
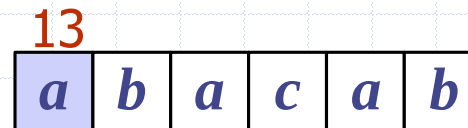
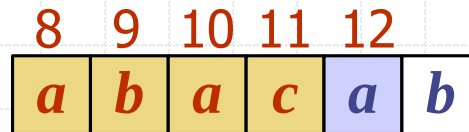
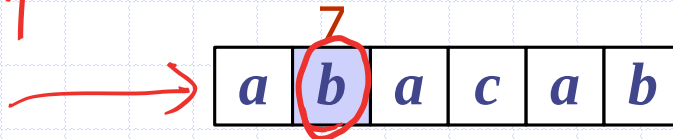
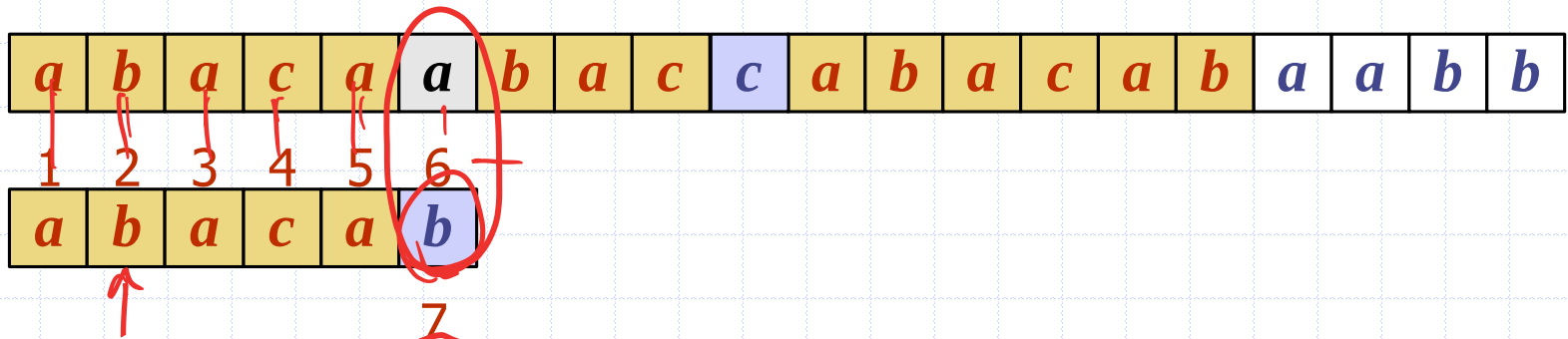
- ◆ Failure function can be represented by an array and can be computed in $O(m)$ time
- ◆ Each iteration of the while-loop, either
 - i increases by one, or
 - shift amount $i - j$ increases by at least one (observe that $F(j - 1) < j$)
- ◆ Hence, there are no more than $2n$ iterations of the while-loop
- ◆ Thus, KMP's algorithm runs in optimal time $O(m + n)$

Algorithm *KMPMatch*(T, P)

```
*  $F \leftarrow \text{failureFunction}(P)$   $O(m)$  *  
 $i \leftarrow 0$   
 $j \leftarrow 0$   
while  $i < \text{length}(T)$   
→ if  $T[i] = P[j]$  then  
    if  $j = \text{length}(P) - 1$  then  
        return  $i - j$  { match }  
    else *  
         $i \leftarrow i + 1$   
         $j \leftarrow j + 1$   
* else  
→ if  $j > 0$  then  
     $j \leftarrow$   $F[j - 1]$   
else  
     $i \leftarrow i + 1$   
return  $-1$  { no match }
```

Example

$0 \ 1 \ 2 \ 3 \ 4 \ 5$
 $a \ b \ a \ c \ a \ b$
 $\uparrow$
 $j = 5$
 $j-1 = 4$



j	0	1	2	3	4	5
$P[j]$	a	b	a	c	a	b
$F(j)$	0	0	1	0	1	2

Week 10 – Strings and Patterns

1. Strings & Pattern Matching
2. Tries → “Retrieval Trees”
3. Suffix Arrays

Preprocessing Strings

- ◆ Preprocessing the pattern speeds up pattern matching queries
 - After preprocessing the pattern, KMP's algorithm performs pattern matching in time proportional to the text size

$$O(m + n)$$

Preprocessing Strings

- ◆ If text is large, and searched often, preprocess the text
 - trie supports pattern matching queries in time proportional to the pattern size

Standard Tries

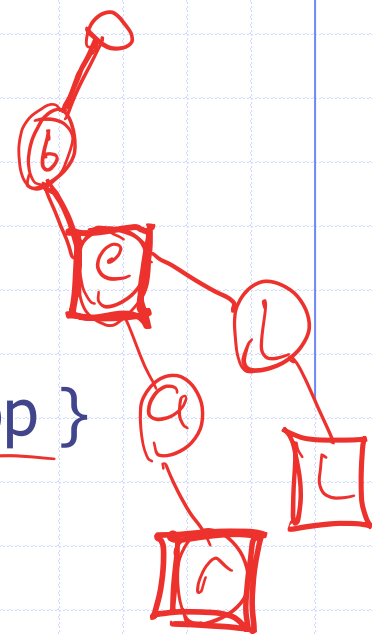
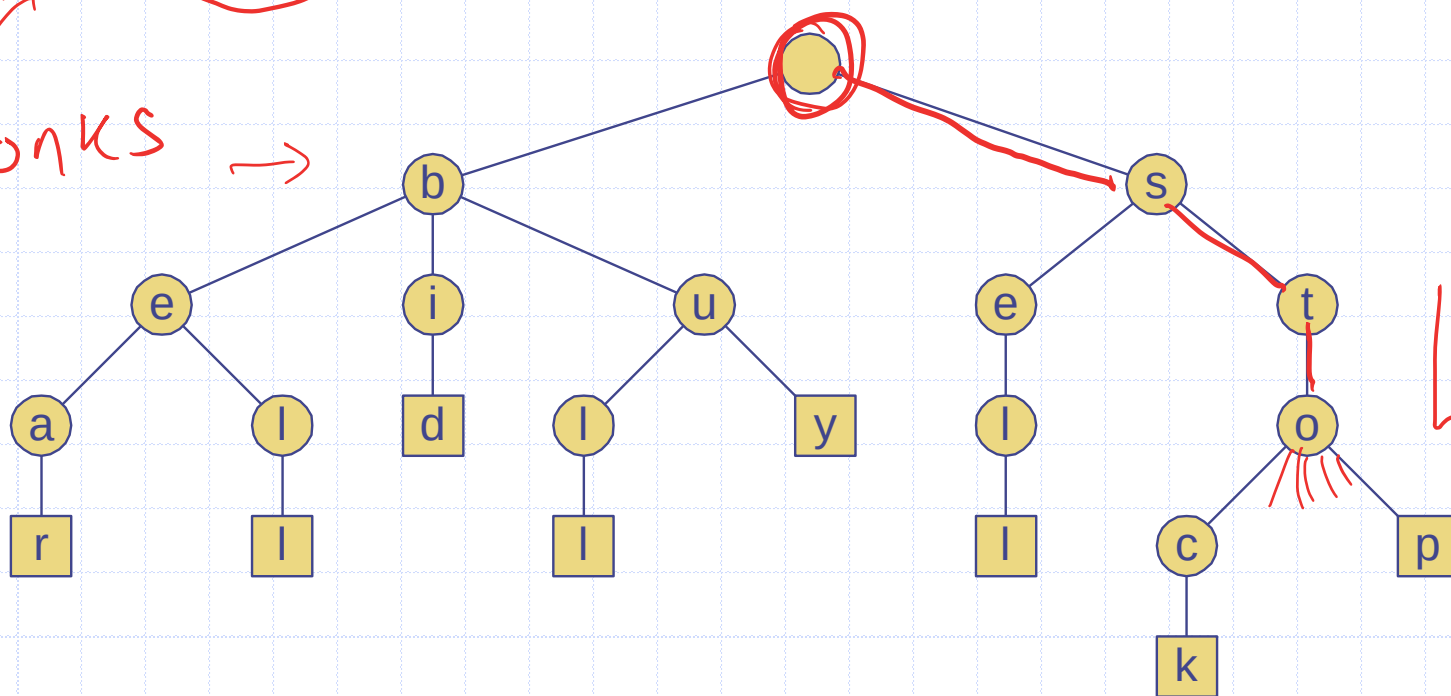
- ◆ Standard trie for a set of strings S is an ordered tree
 - each node, but the root, is labelled with a character
 - children of a node are alphabetically ordered
 - paths from the root to the external nodes yield the strings of S

Example Standard Trie

◆ Standard trie for the set of strings

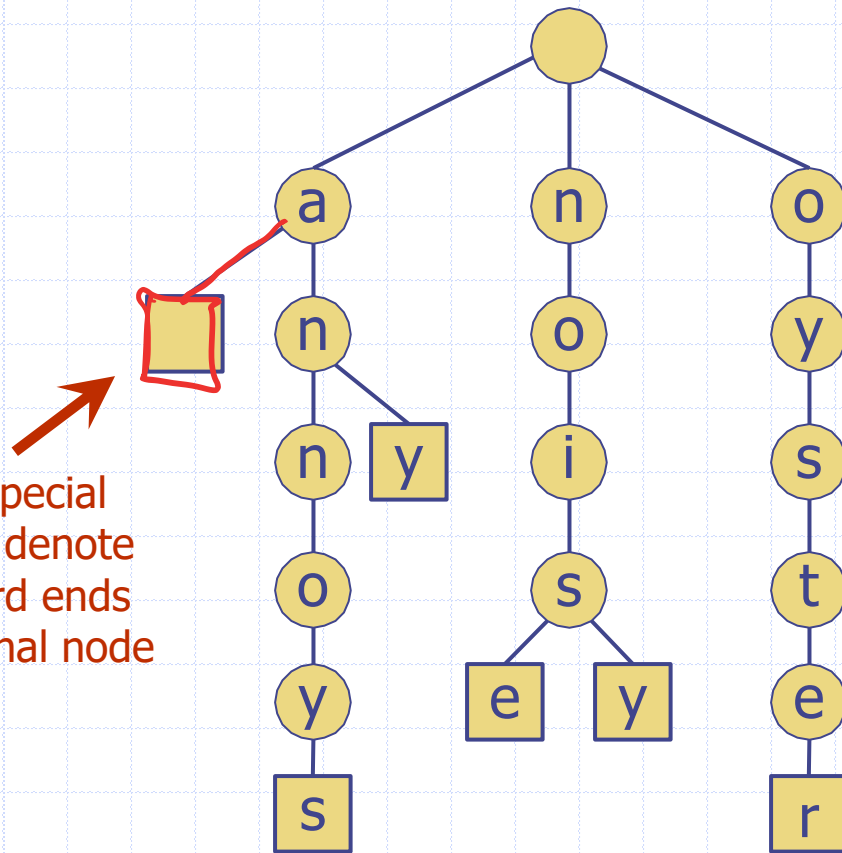
S = { bear, bell, bid, bull, buy, sell, stock, stop }

Stocks →



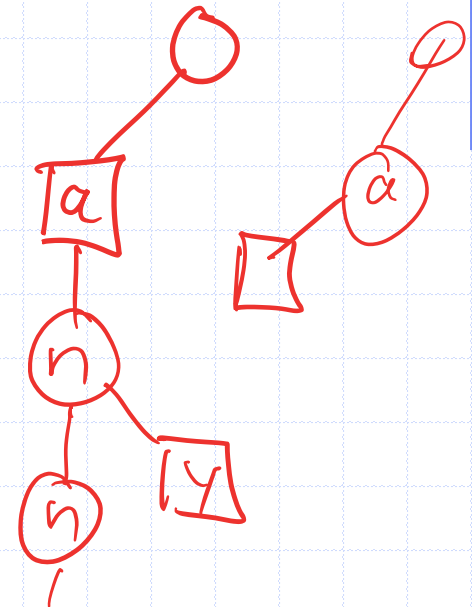
Create a standard trie for the set of strings

$S = \{\text{any, noise, annoys, a, noisy, oyster}\}$

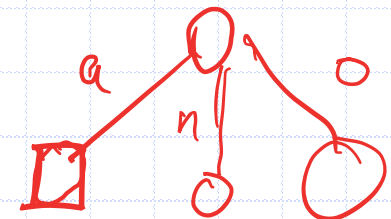


Need a special symbol to denote that a word ends on an internal node

Careful 1



Careful 2



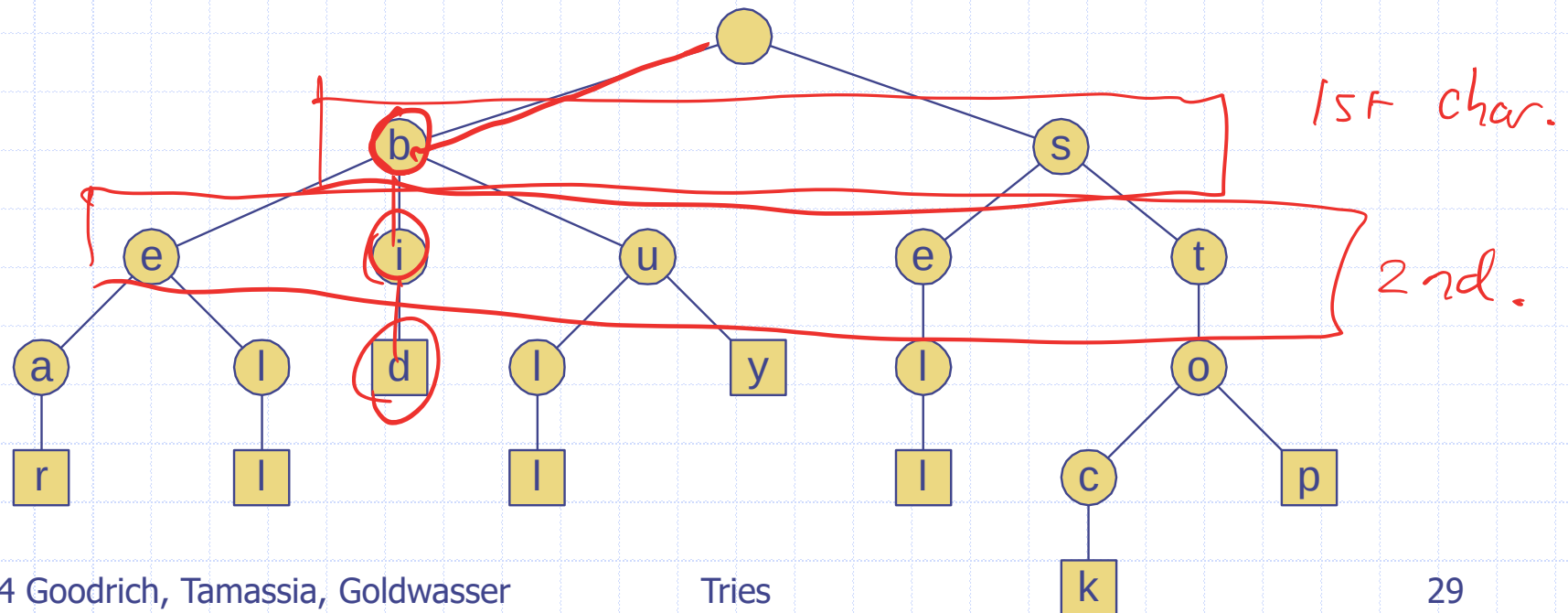
Analysis of Standard Tries

n total size of the strings in S

m size of the string parameter of the (e.g. search) operation

d size of the alphabet

b i d



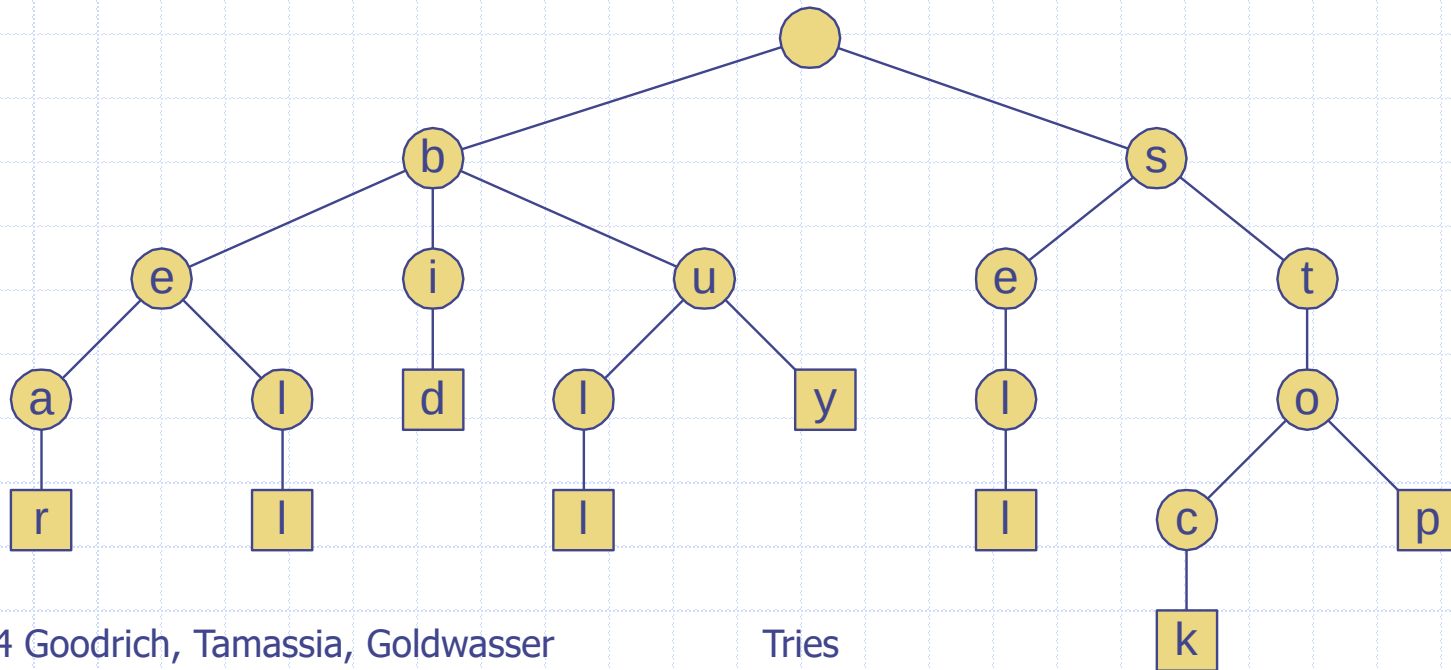
Analysis of Standard Tries

n total size of the strings in S

m size of the string parameter of the (e.g. search) operation

d size of the alphabet

◆ uses $O(n)$ space



Analysis of Standard Tries

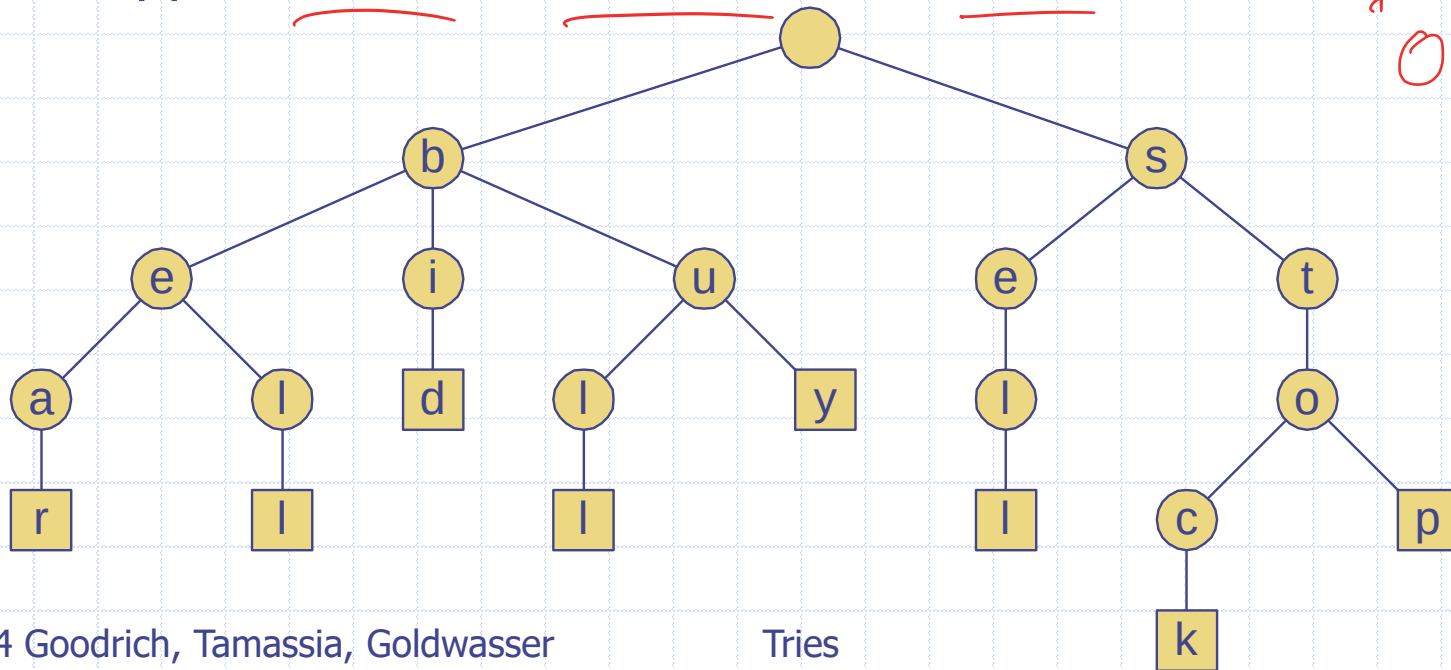
n total size of the strings in S

m size of the string parameter of the (e.g. search) operation

d size of the alphabet (perhaps fixed? $\{A, G, T, C\}$)

◆ uses $O(n)$ space

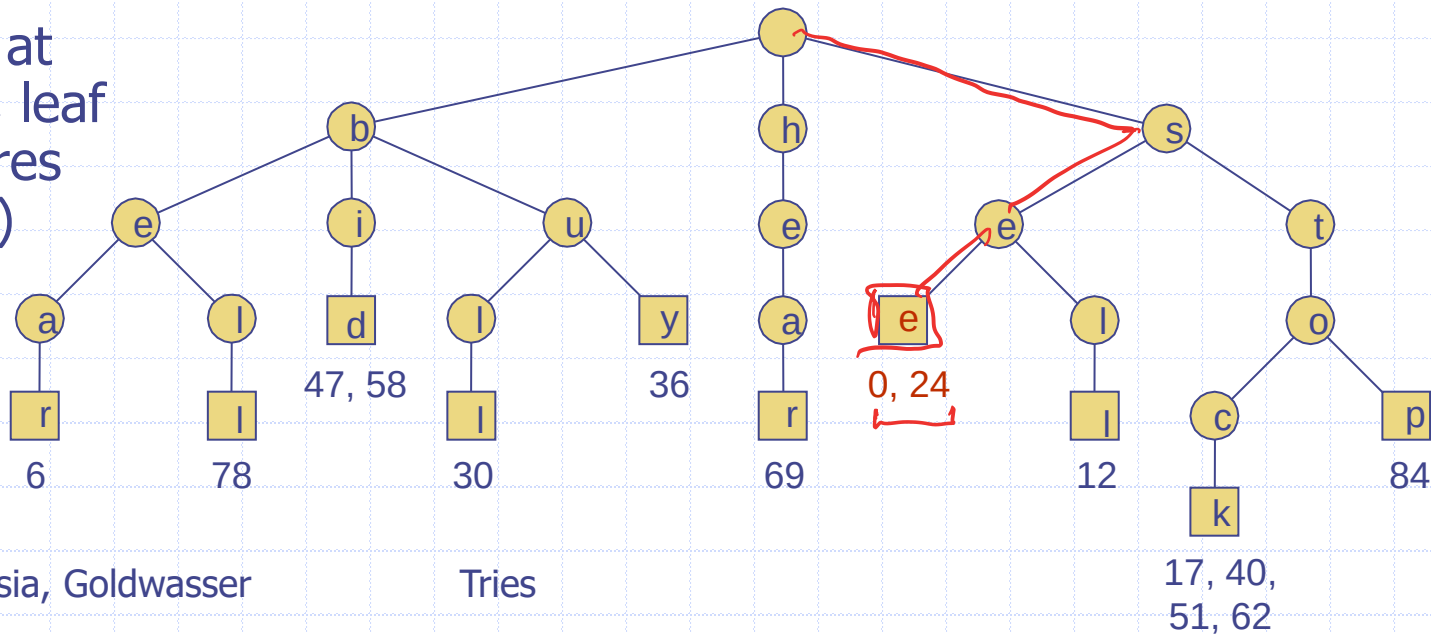
◆ supports searches, insertions and deletions in time $O(dm)$
 $O(m)$



Word Matching with a Trie

- ◆ Insert words of the text into trie
- ◆ Each leaf is associated w/ one particular word
- ◆ Leaf stores indices where associated word begins
("see" starts at index 0 & 24, leaf for "see" stores those indices)

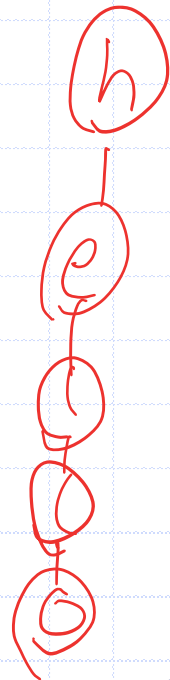
s	e	e		a		b	e	a	r	?		s	e	l	l		s	t	o	c	k	!	
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23
s	e	e		a		b	u	l	l	?		b	u	y		s	t	o	c	k	!		
24	25	26	27	28	29	30	31	32	33	34	35	36	37	38	39	40	41	42	43	44	45	46	
b	i	d		s	t	o	c	k	!		b	i	d		s	t	o	c	k	!			
47	48	49	50	51	52	53	54	55	56	57	58	59	60	61	62	63	64	65	66	67	68		
h	e	a	r		t	h	e		b	e	l	l	?		s	t	o	p	!				
69	70	71	72	73	74	75	76	77	78	79	80	81	82	83	84	85	86	87	88				



Compressed Tries

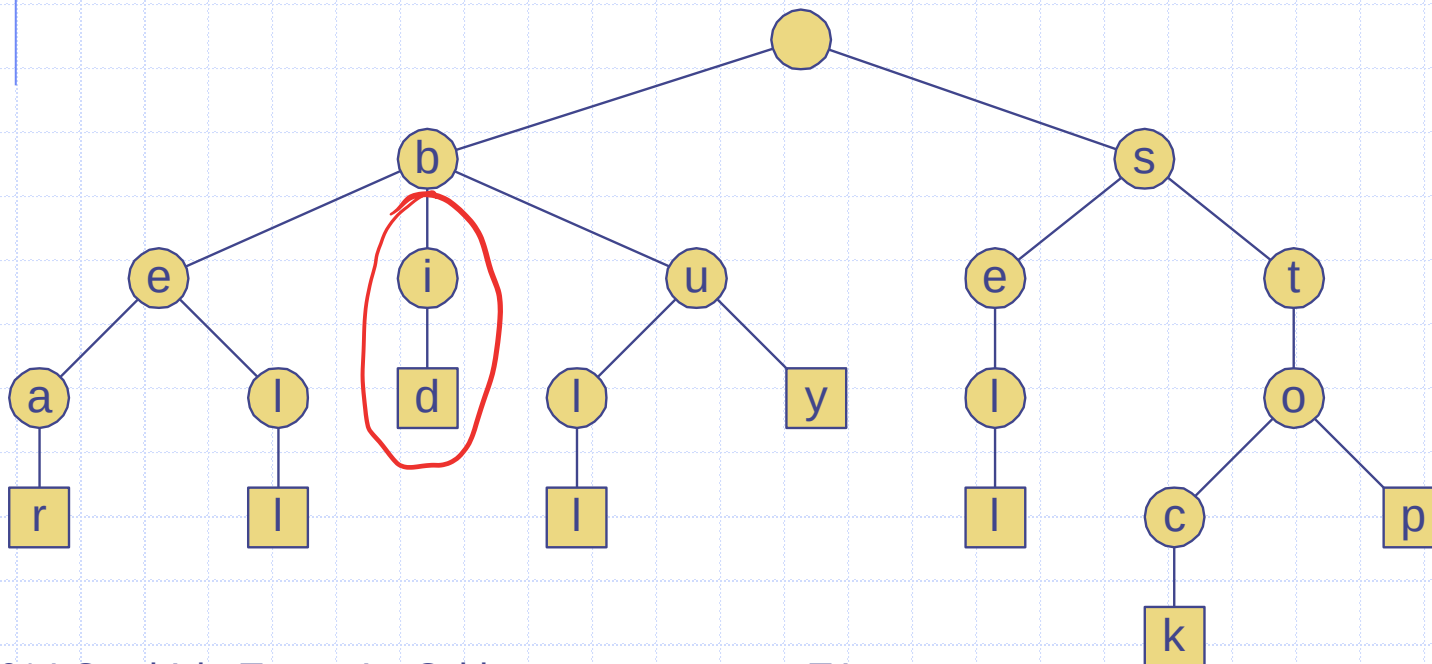
- ◆ Compressed trie has internal nodes of degree at least two
- ◆ Obtained from standard trie by compressing chains of *redundant* nodes

hello



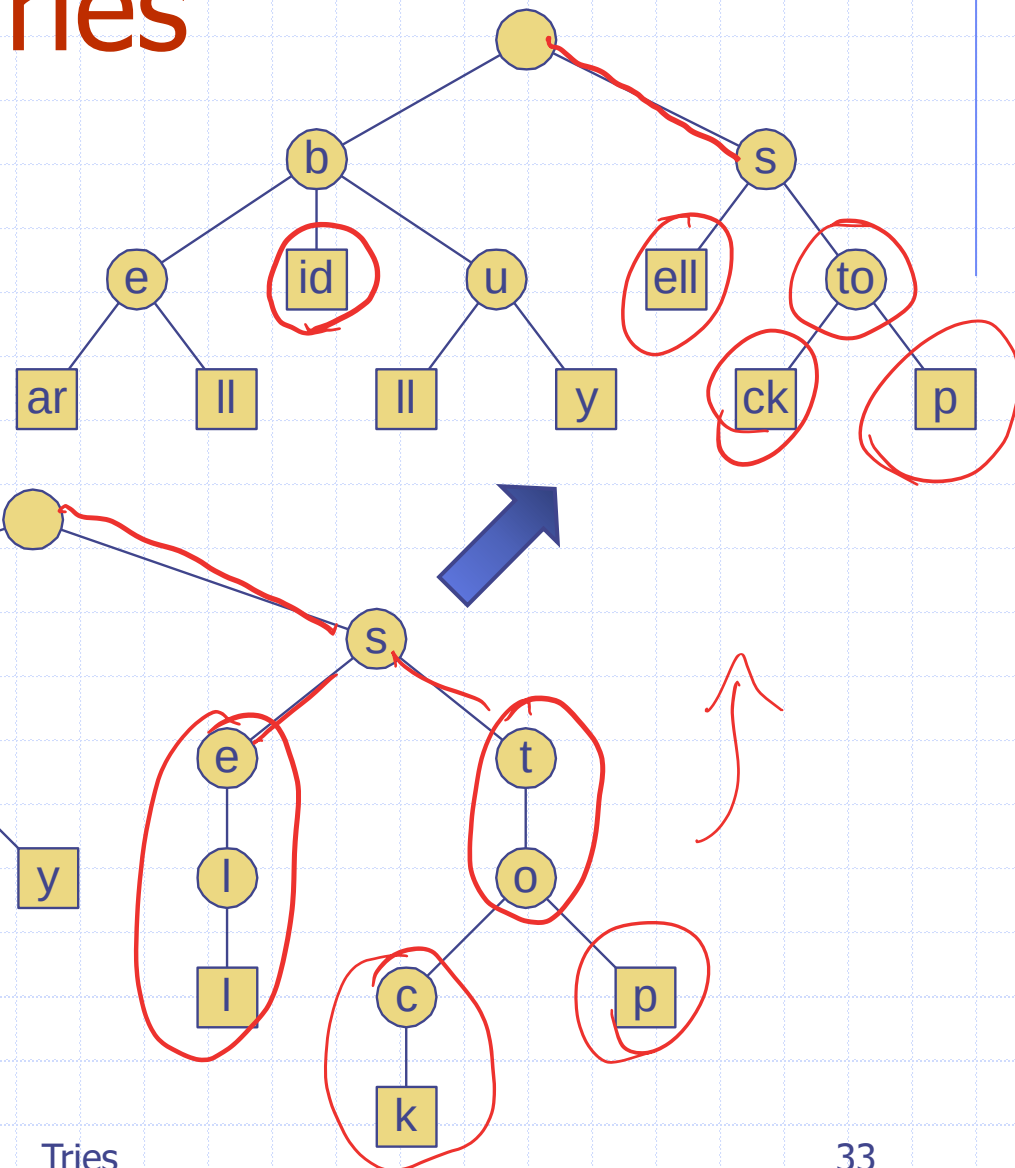
Compressed Tries

- ◆ “i” and “d” in “bid” are *redundant*
 - they signify the same word



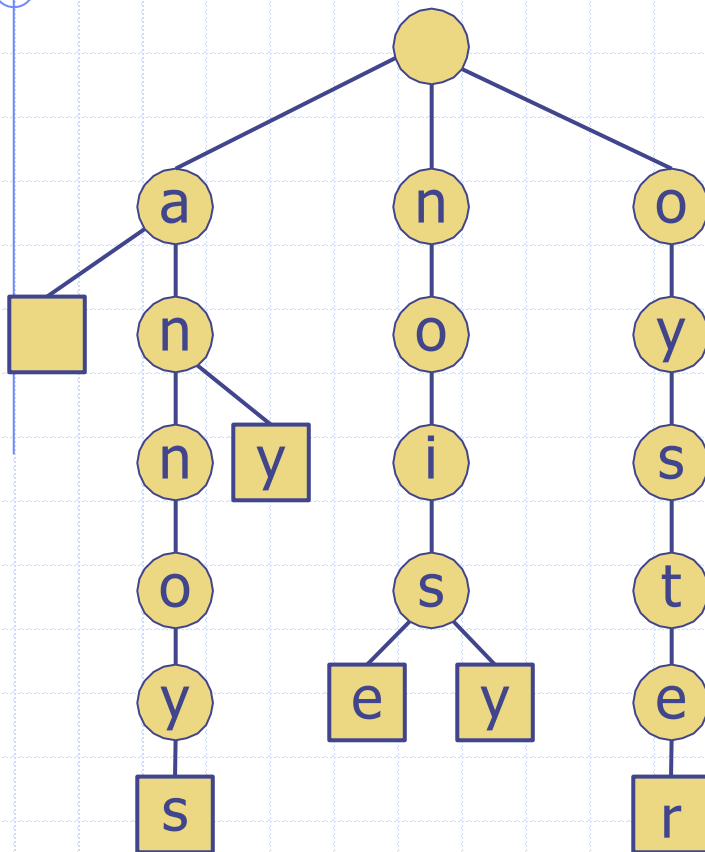
Compressed Tries

- ◆ “i” and “d” in “bid” are *redundant*
 - they signify the same word



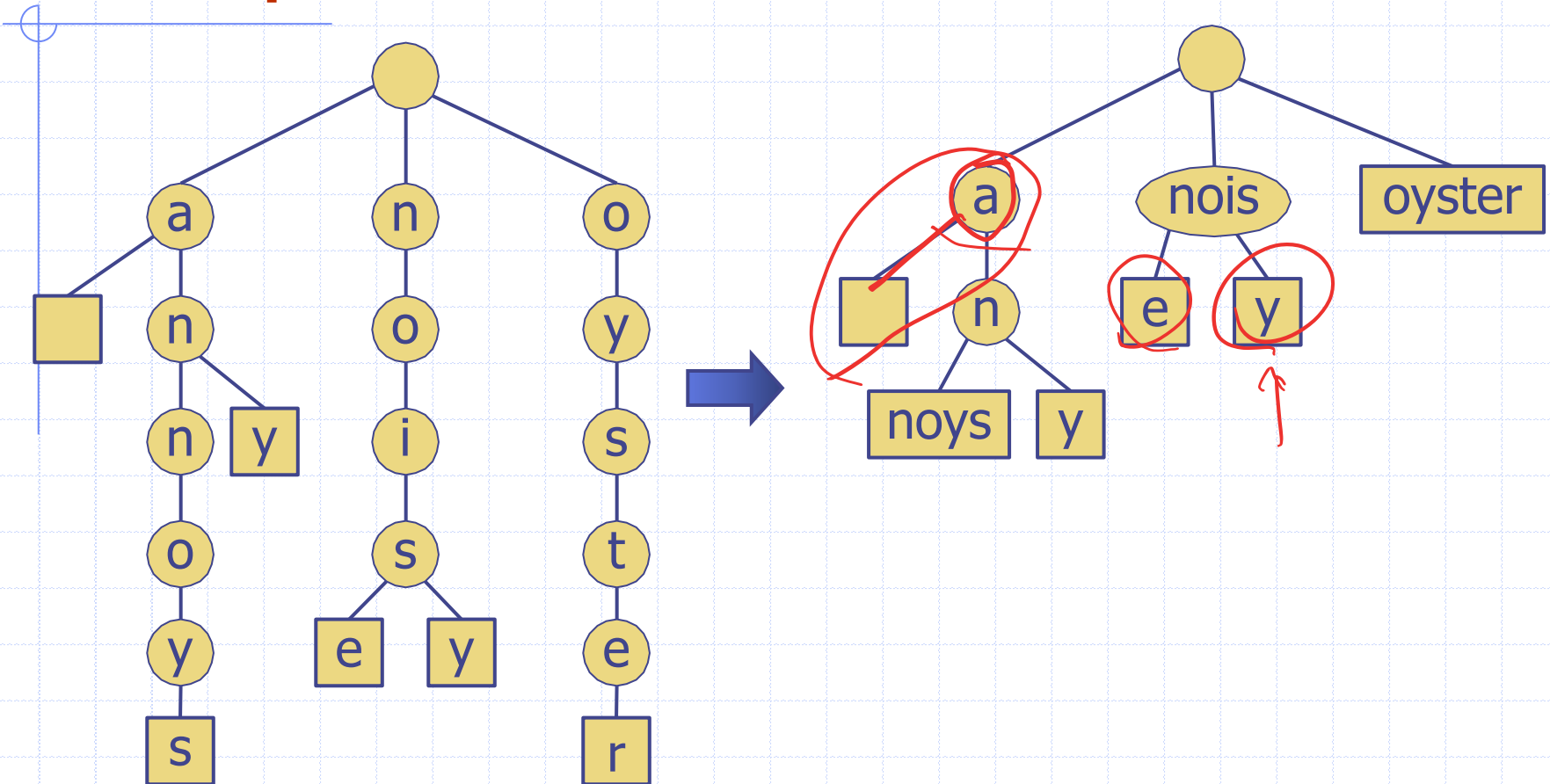
$S = \{\text{any, noise, annoys, a, noisy, oyster}\}$

Compressed Tries



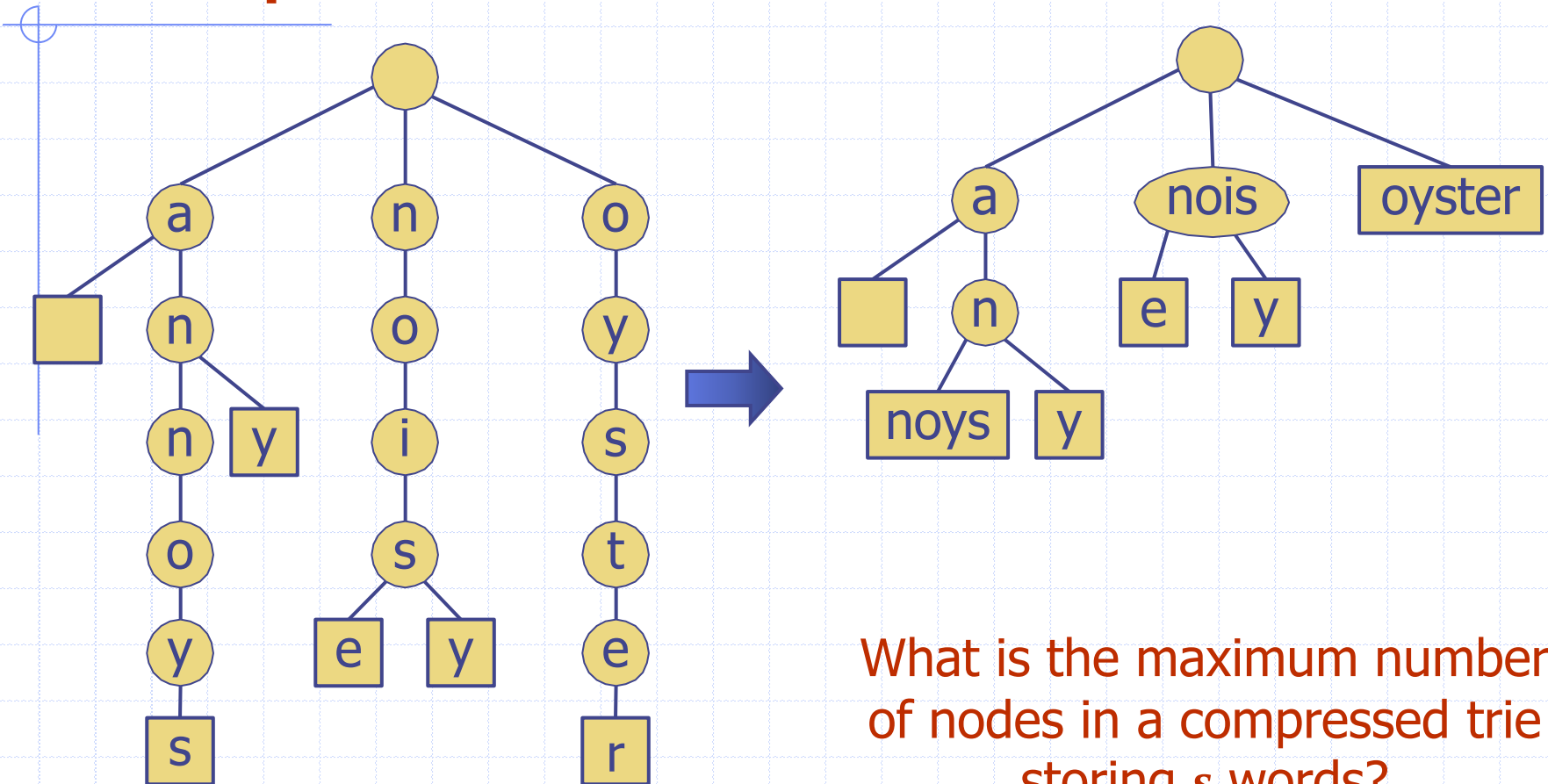
$S = \{\text{any, noise, annoys, a, noisy, oyster}\}$

Compressed Tries



$S = \{\text{any, noise, annoys, a, noisy, oyster}\}$

Compressed Tries

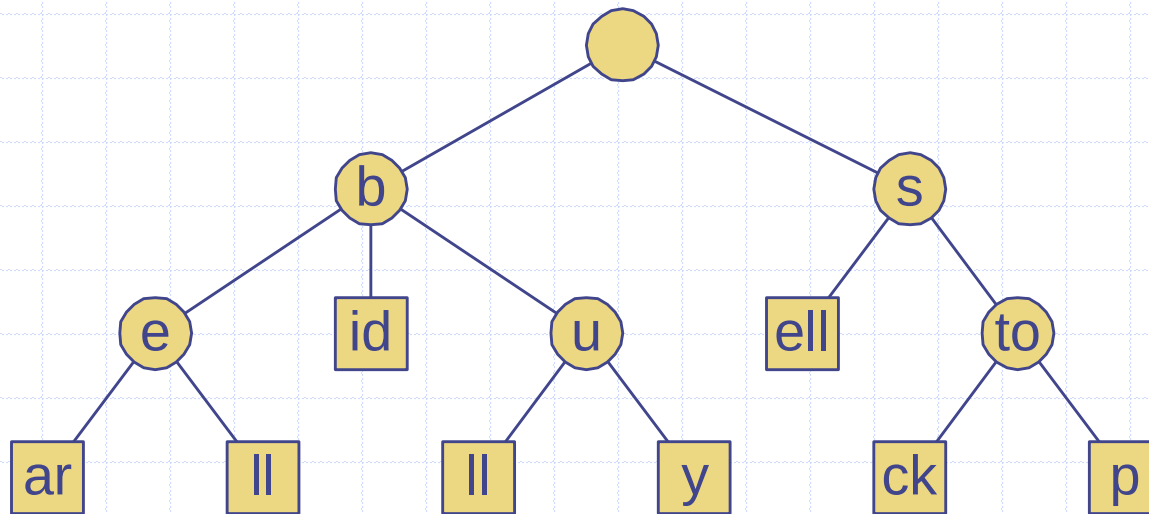


What is the maximum number of nodes in a compressed trie storing s words?

$$\underline{s + (s - 1) = 2s - 1}$$

Compact Representation

- ◆ Want to create a compact representation of a compressed tree

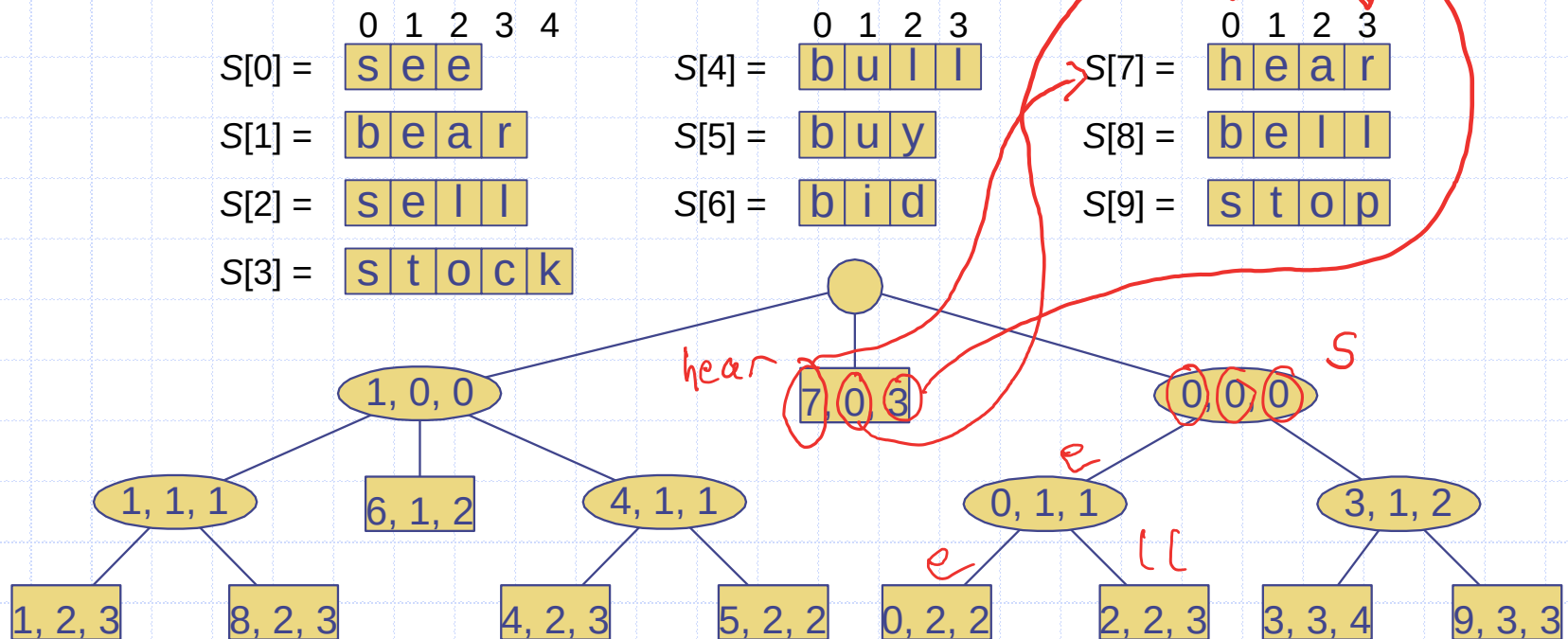


Sorry for not preparing properly, my bad! Below, each node in the ~~try~~ is an [index, start, end] tuple

Compact Representation

◆ Compact representation of a compressed trie for an array of strings

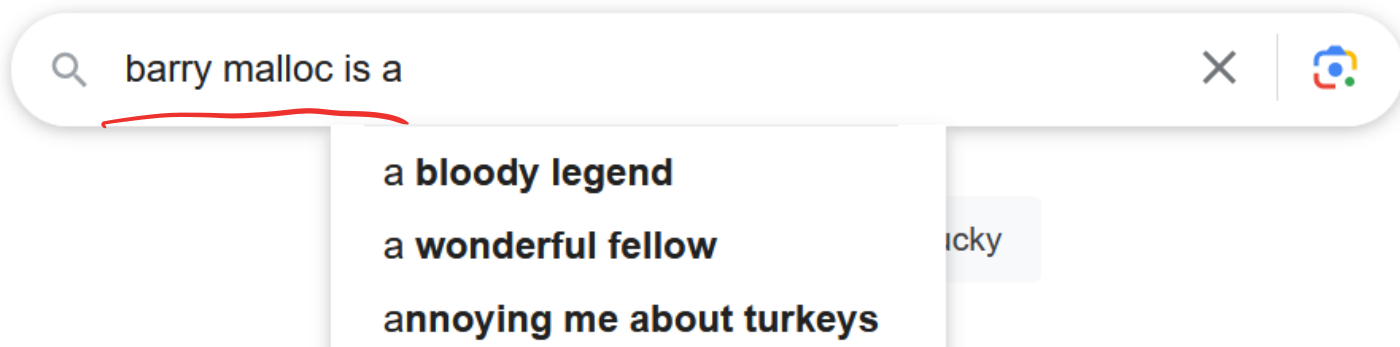
- Stores ranges of indices instead of substrings at the nodes
- Uses $O(s)$ space, where s is the number of strings in the array
- Serves as an auxiliary index structure



Tries: Outside of Patterns

- ◆ Tries have other common uses outside of simple pattern matching (finding patterns in a given text efficiently)
- ◆ One common example: Autosuggestion engines

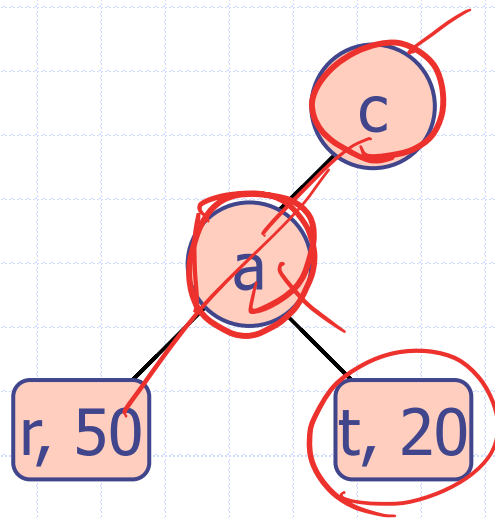
Query Autosuggestion



- "brisbane weather"

Building the Trie

- Each node (or edge) also holds a **weight** corresponding to the volume of queries that were issued for that prefix



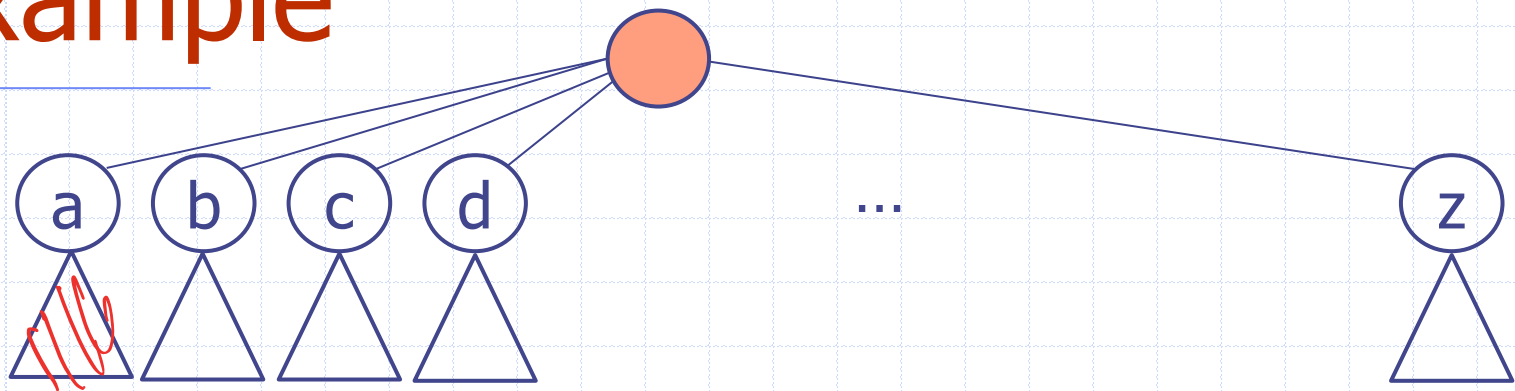
cat (20)
car (50)

ca

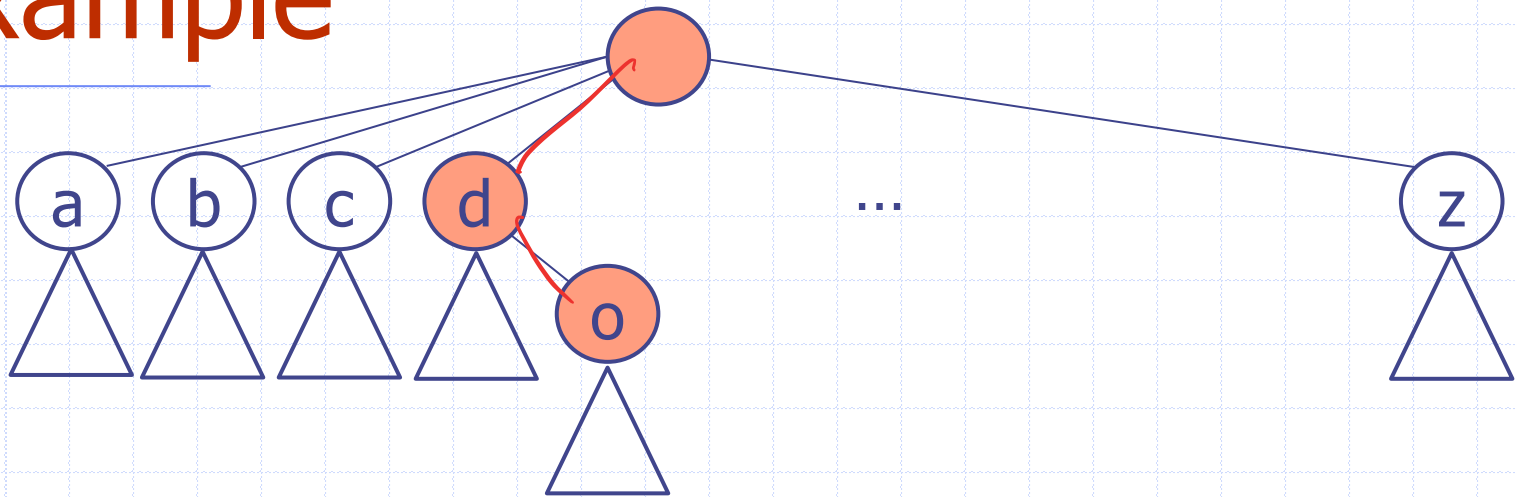
Querying the Trie

- Given a prefix P , we want to return a ranked list of k completions
 - k is usually small, about 5-10 suggestions
 - P is usually streamed, increasing in length
 - ◆ Query starts empty, grows character-by-character
 - ◆ Search is repeatedly triggered as the query is built, and suggestions returned ASAP!
 - ◆ Only return endpoint suggestions – ranked by weight

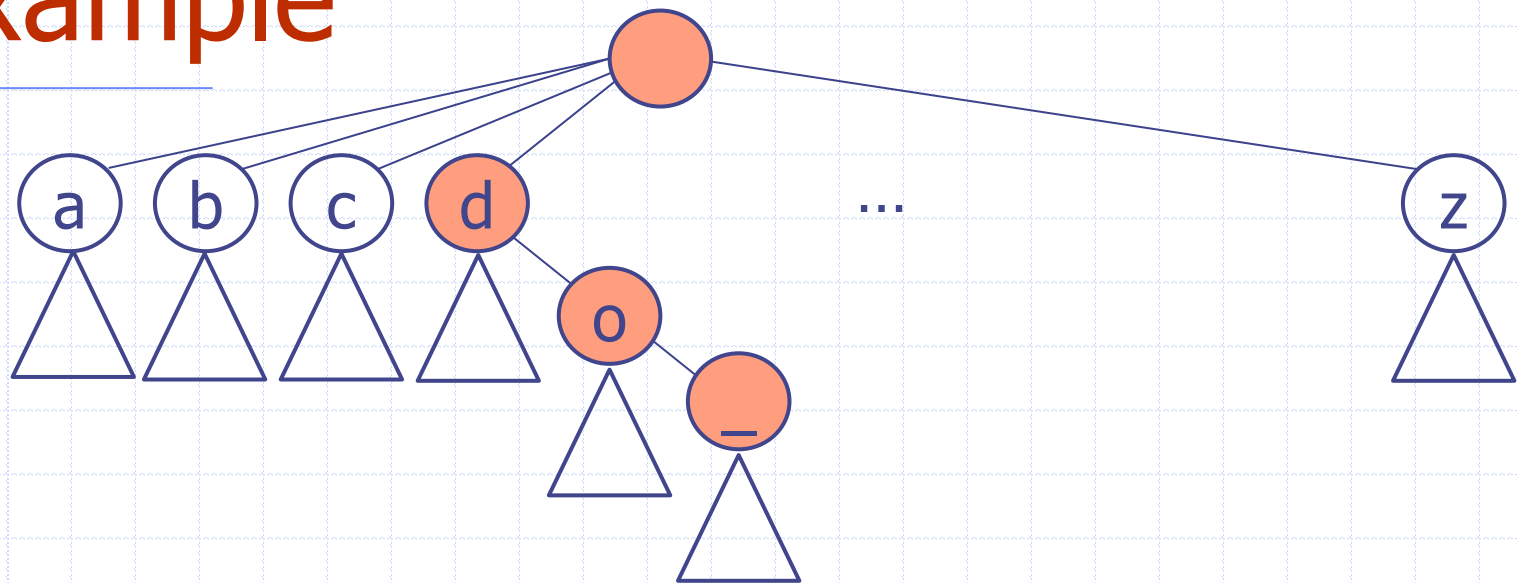
Example



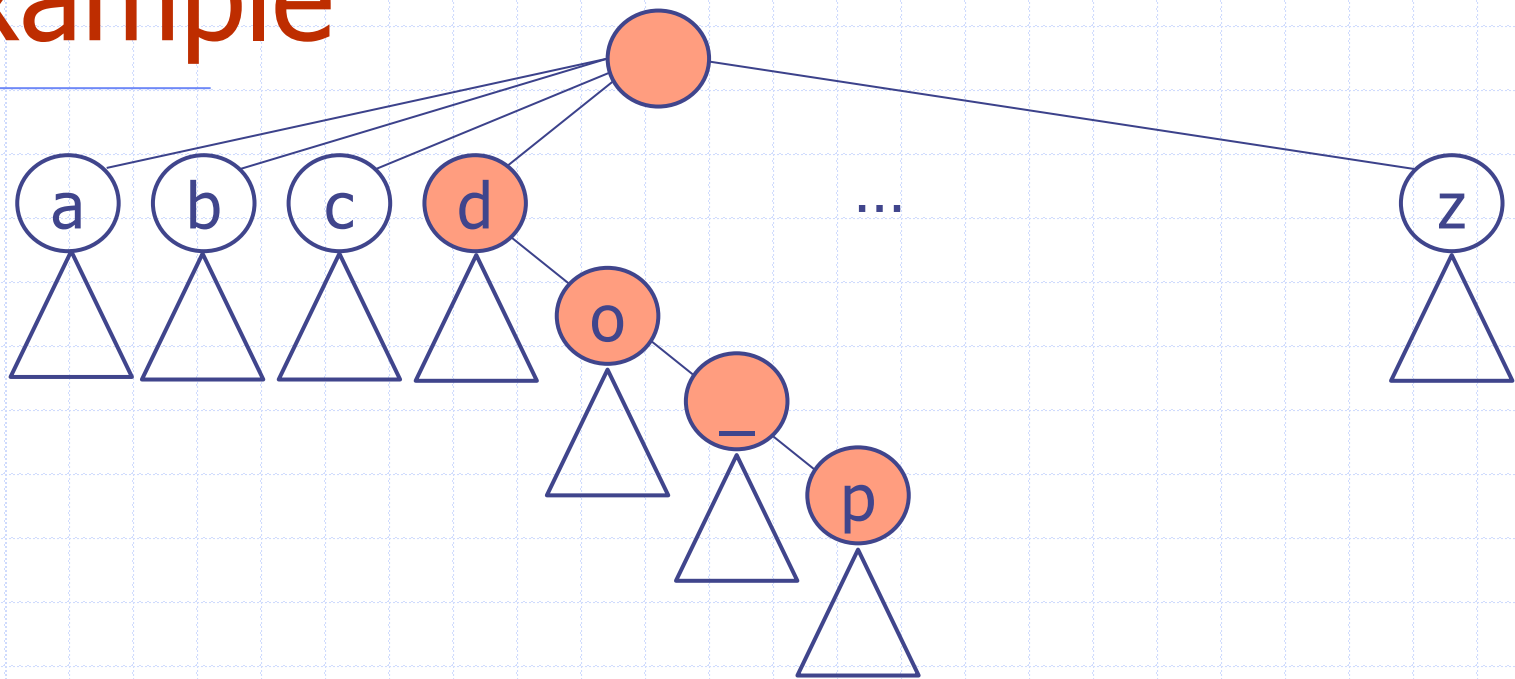
Example



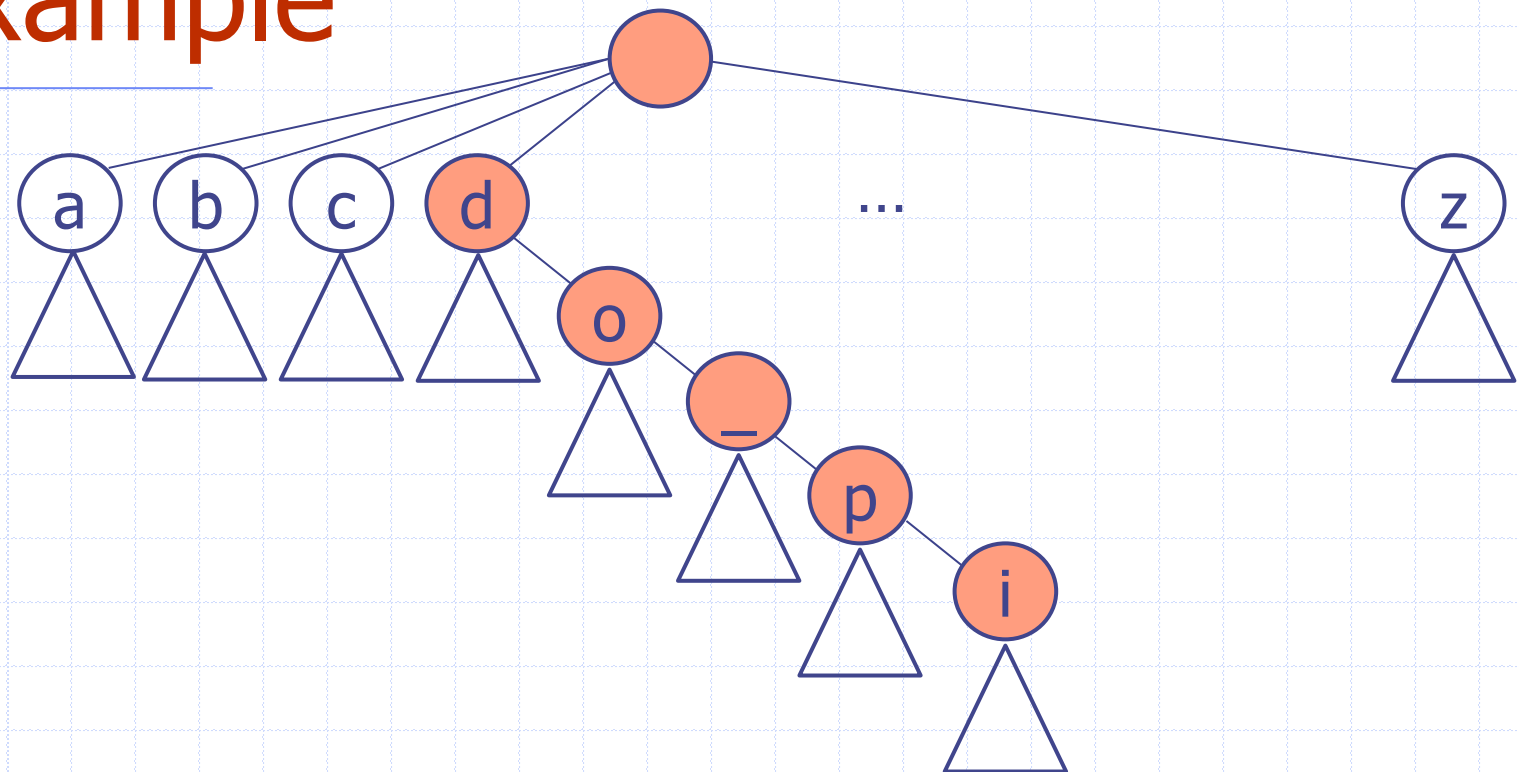
Example



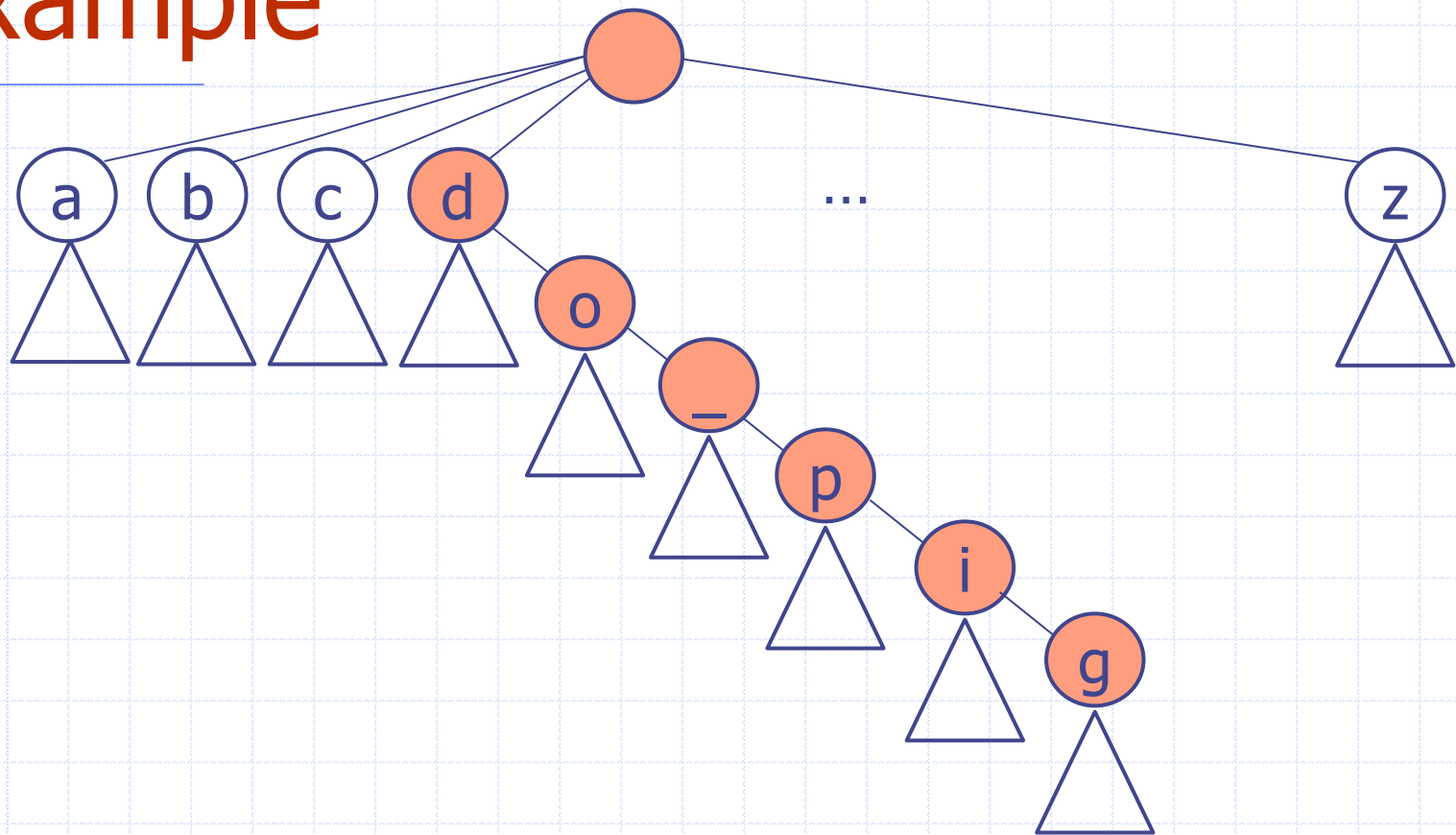
Example



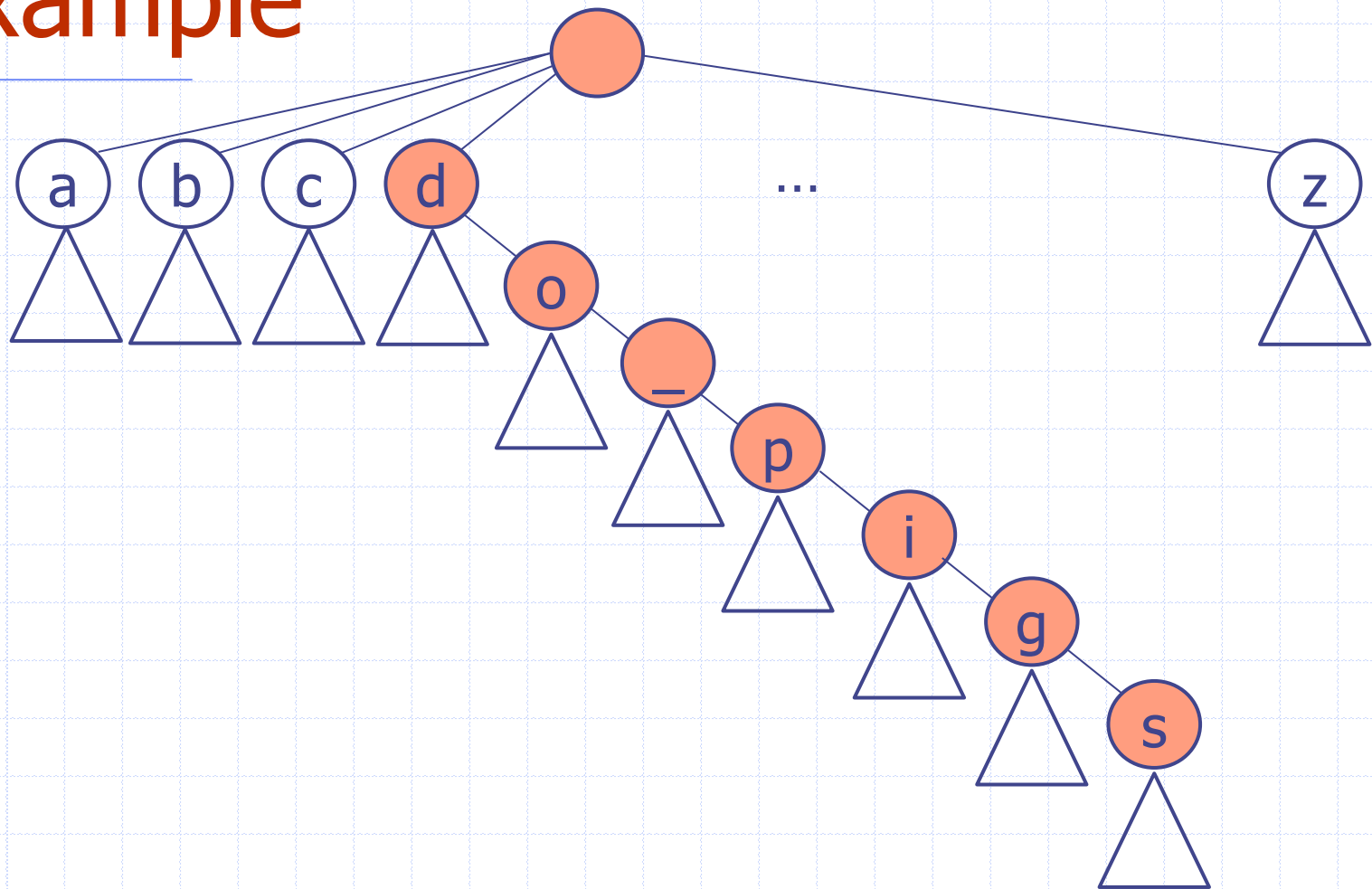
Example



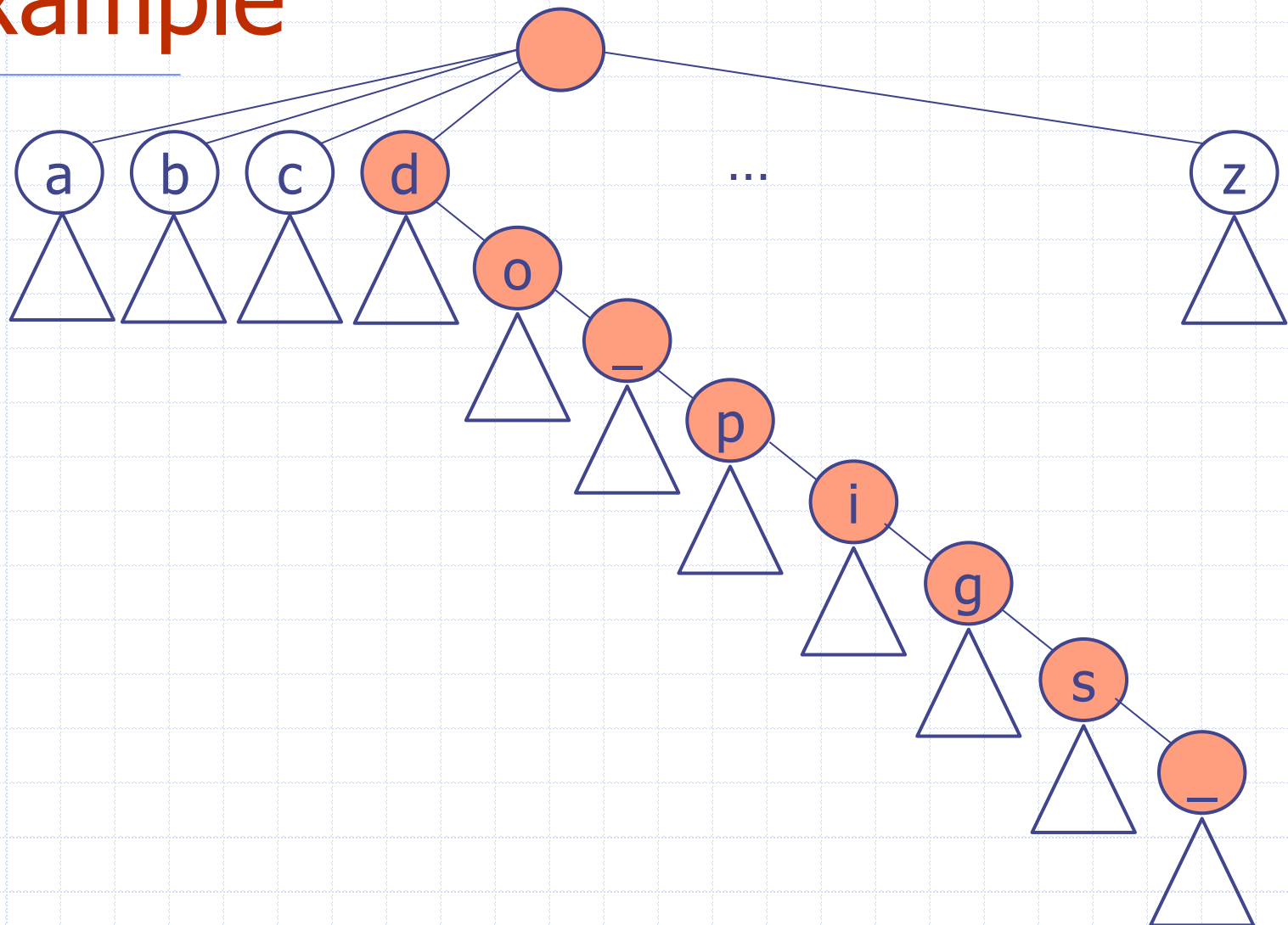
Example



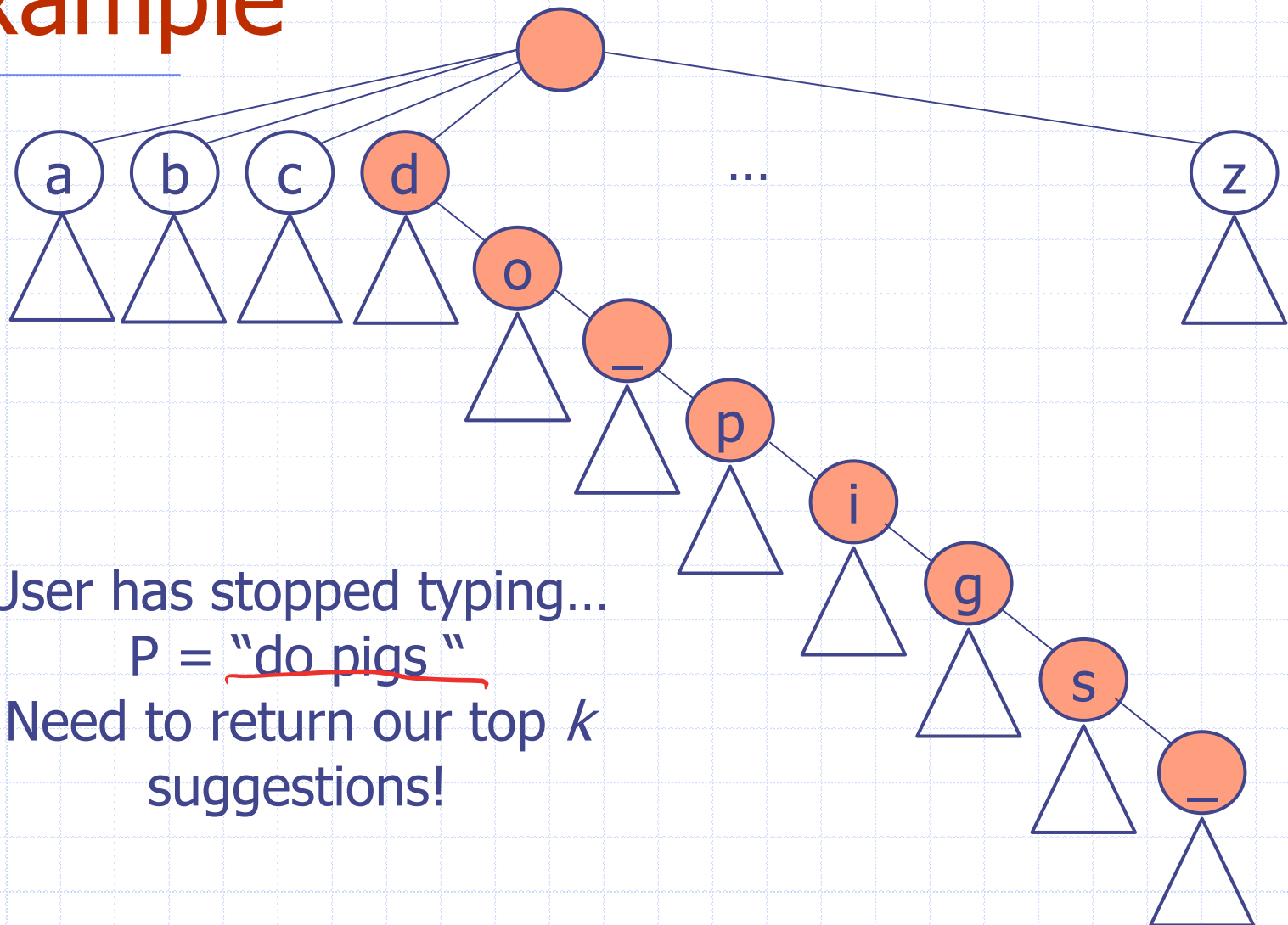
Example



Example



Example

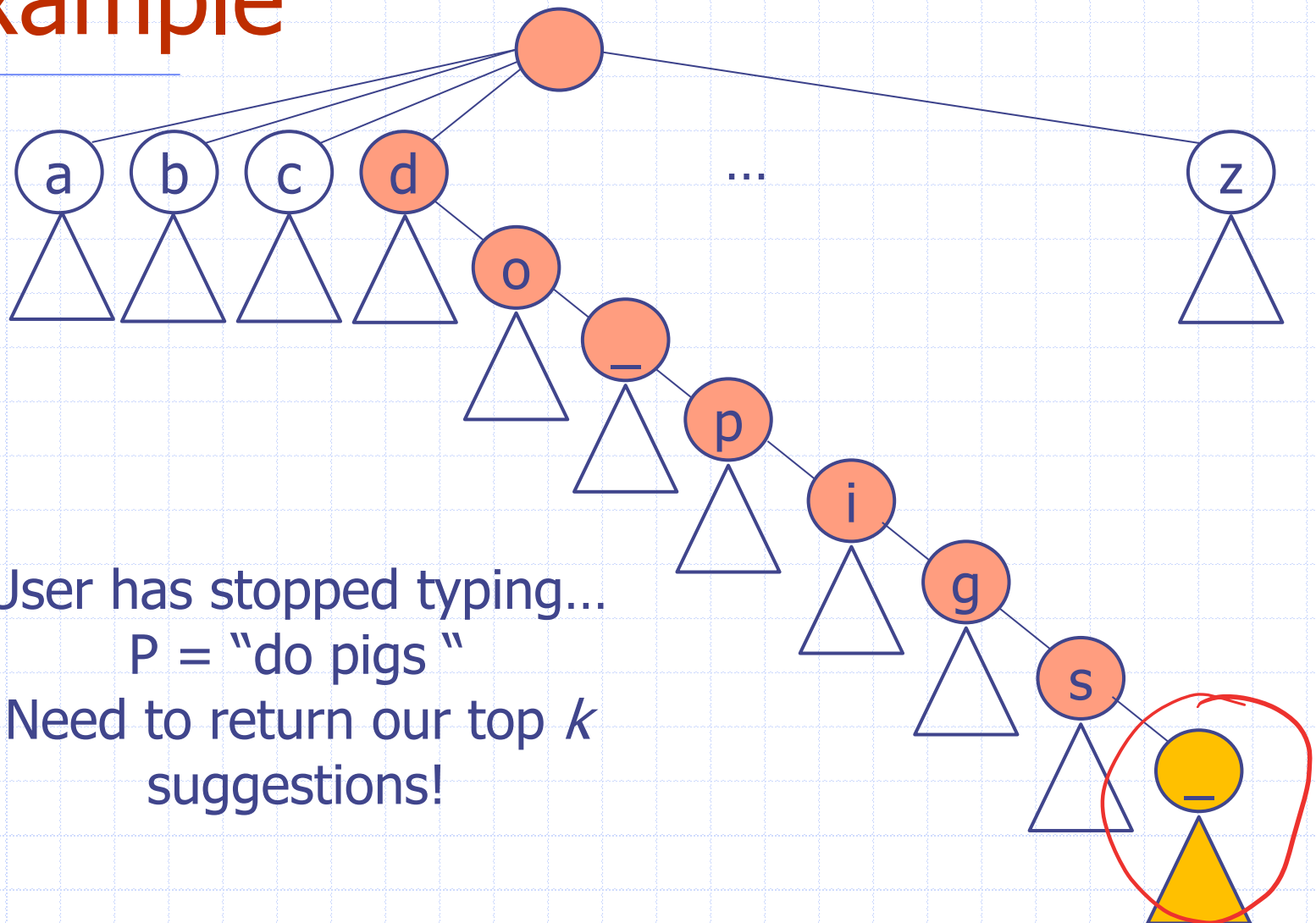


User has stopped typing...

P = "do pigs"

Need to return our top k suggestions!

Example



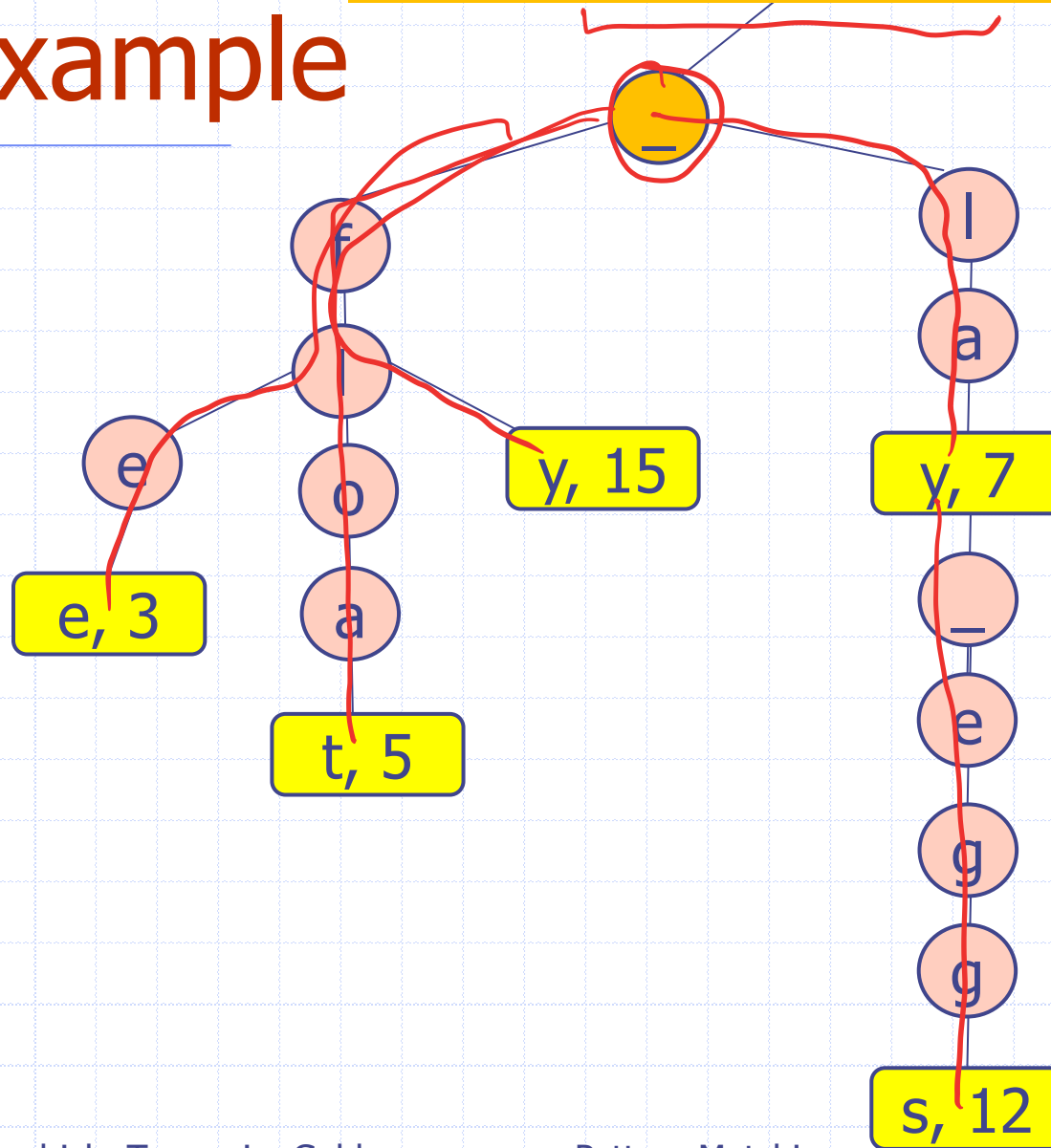
User has stopped typing...

P = "do pigs "

Need to return our top k suggestions!

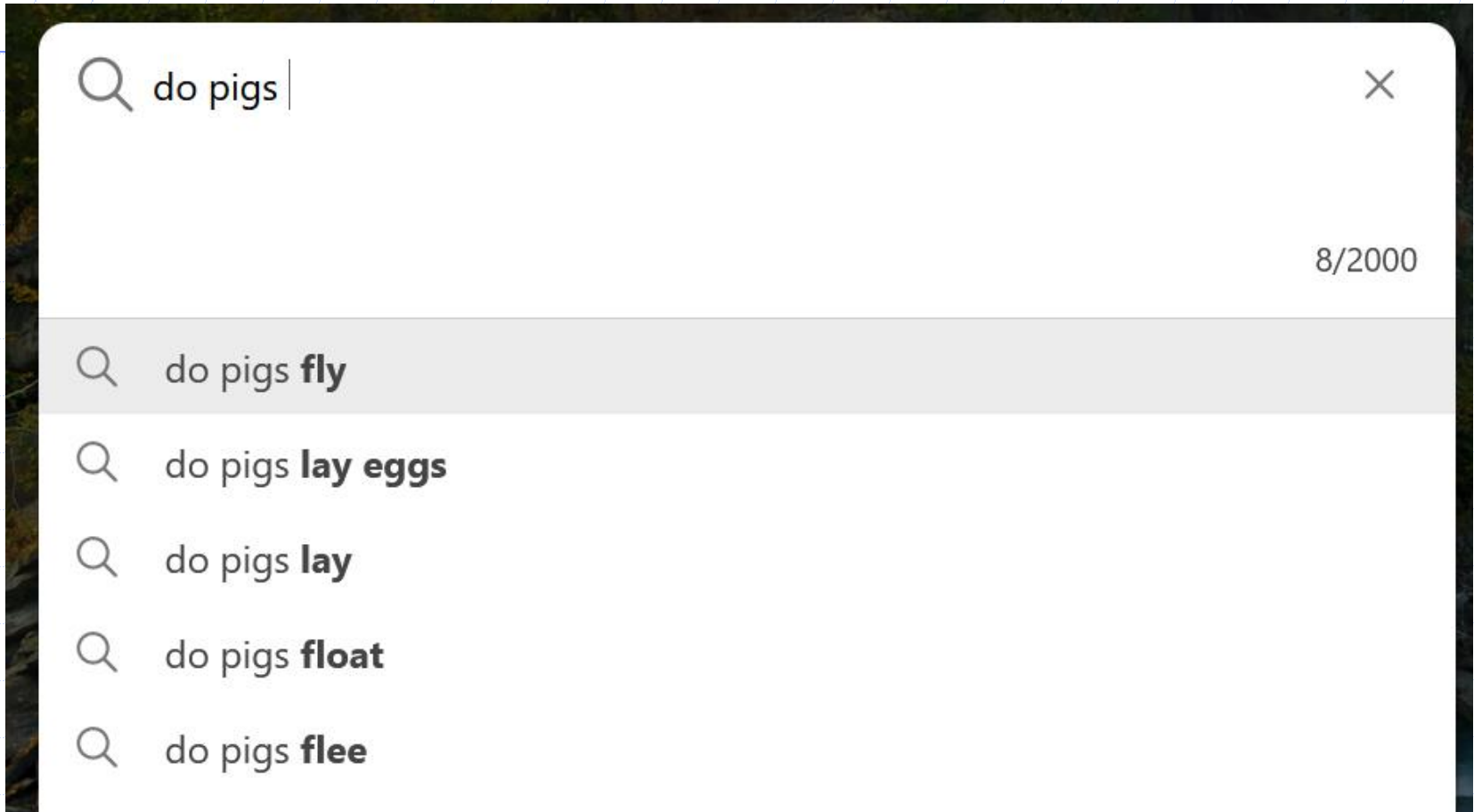
P = "do pigs "

Example



float, 5
fly, 15
flee, 3
lay, 7
lay eggs, 12

MallocSearch TM



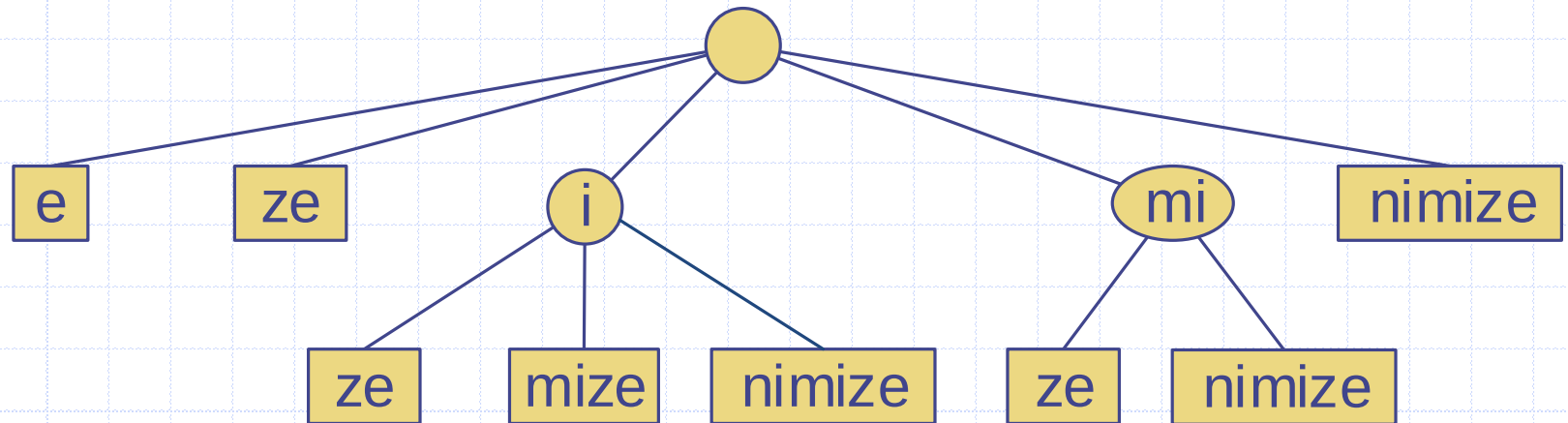
Week 10 – Strings and Patterns

1. Strings & Pattern Matching
2. Tries
3. Suffix Arrays

Suffix Tree (Suffix Trie)

- ◆ Suffix tree of a string T is the compressed trie of all the suffixes of T

m	i	n	i	m	i	z	e
0	1	2	3	4	5	6	7



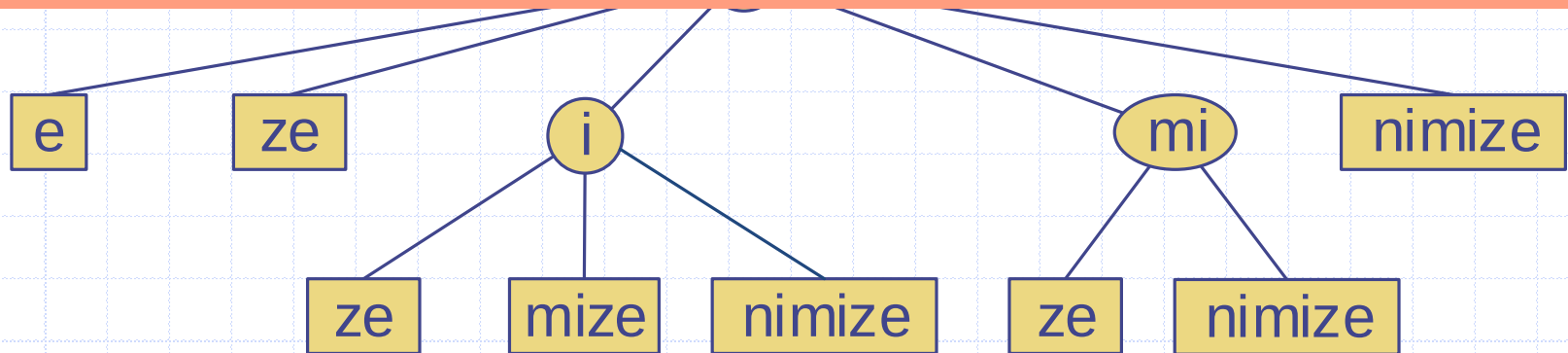
Suffix Tree (Suffix Trie)

- ◆ Suffix tree of a string T is the compressed trie of **all** the **suffixes** of T

m	i	n	i	m	i	z	e
0	1	2	3	4	5	6	7

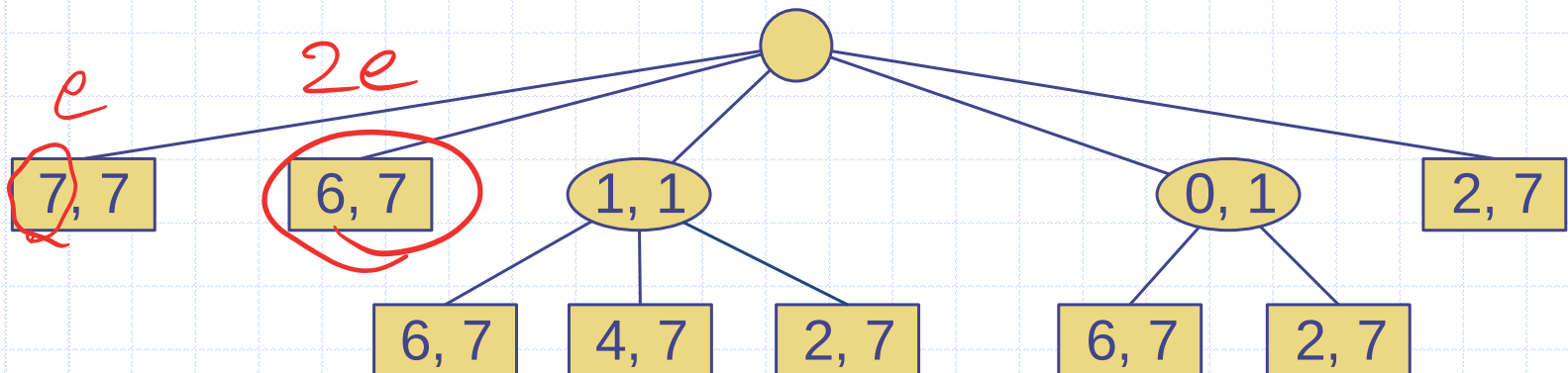
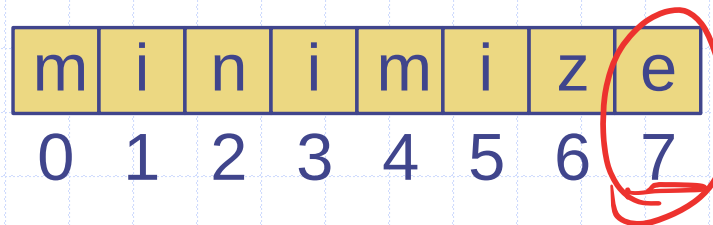
abc
abc
bc
abcd
abcd
bcd
cd
d

How many suffixes are there for a string of length n ?



Suffix Tree: Compact Rep.

- ◆ Suffix tree of a string T is the compressed trie of all the suffixes of T

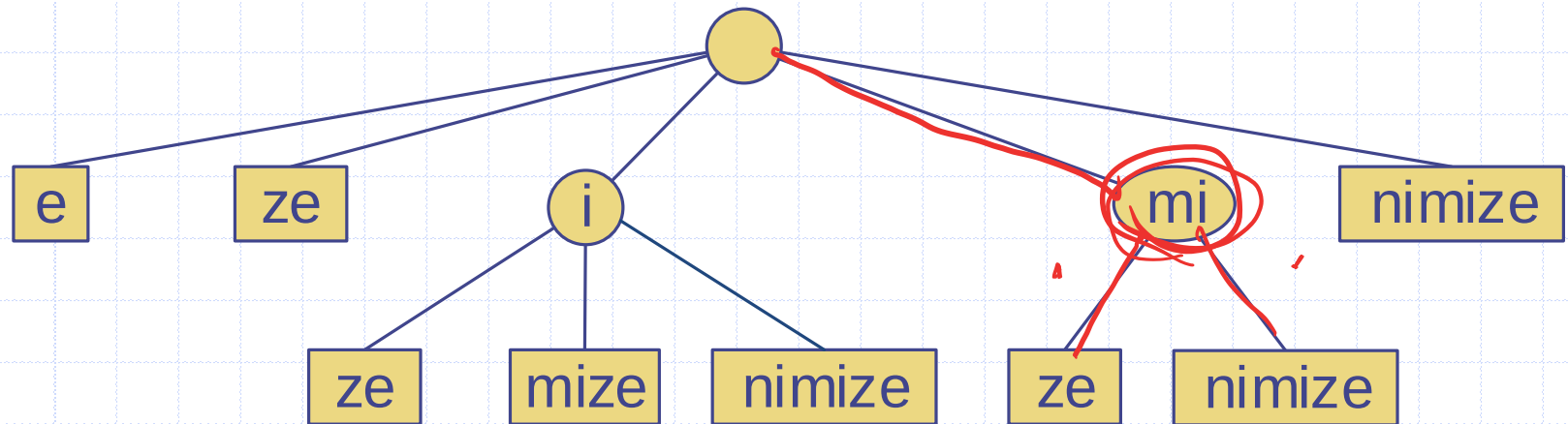


Suffix Tree Pattern Matching

- Maintain, at each node, the number of leaf nodes below
- Walk down the tree until we run out of pattern to match
- Return the count of leaf nodes below us

Pattern: "mi"

minimize
0 4



mize

Minimize

Other Query Types

- Count the number of occurrences of P in T (we just saw this)
- Output the locations of all occurrences of P in T
- Check whether P is a suffix of T
- Find the longest repeat in T (the longest substring that occurs twice in T)
- ...

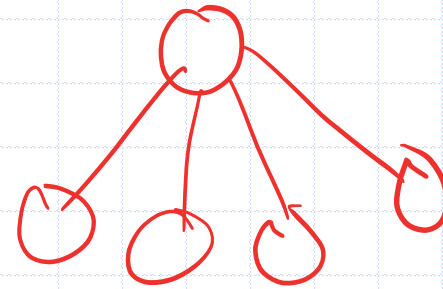
More Query Types...

- Given two strings A and B , what is their longest common substring?
 - Idea: Create a new string by concatenating A and B together with a special delimiter
 - $C = A|B$ (where $|$ is our special delimiter)
 - Build a suffix tree on C
 - Mark subtrees with a bit corresponding to whether the subtree holds a suffix of A
 - Do the same for B
 - The deepest node with both bits set is the longest common substring of both!

Analysis of Suffix Trees

- Compact representation of the suffix tree for a string T of size t from an alphabet of size d
- Uses $O(t)$ space (linear in the input text)
- Supports arbitrary pattern matching queries in T in $O(d \times p)$ time, where p is the size of the pattern
 - If we assume d is $O(1)$, this becomes $O(p)$ time ☺
 - Then, finding all occurrences of P in T will cost $O(p + k)$ where k is the number of times P occurs in T !
- Can construct a suffix tree in $O(t)$ time
 - Out of scope of this course: Ukkonen's algorithm.

Suffix Arrays



- Suffix Trees:

Even though space consumption is $O(t)$ where $t = |\mathcal{T}|$ (linear in the size of the indexed text), their pointer-based representation can be quite costly (Think about very large input strings...)

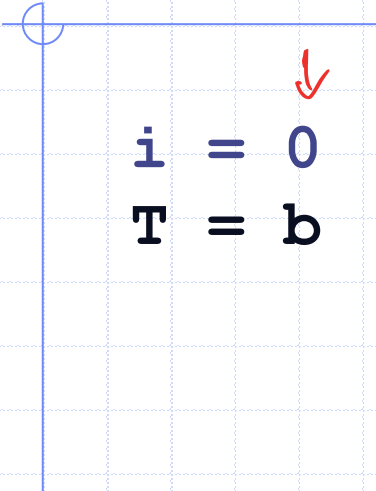
- Idea: We can use an *array* instead!

Suffix Array

0 1
hello ello
 2
 110 ↓
 3
 10

- Write down all suffixes of T
 - The i -th suffix begins at position i
 - Sort the suffixes lexicographically, and let the i values “come along for the ride”
 - The resultant indexes are the suffix array!

Suffix Array Example

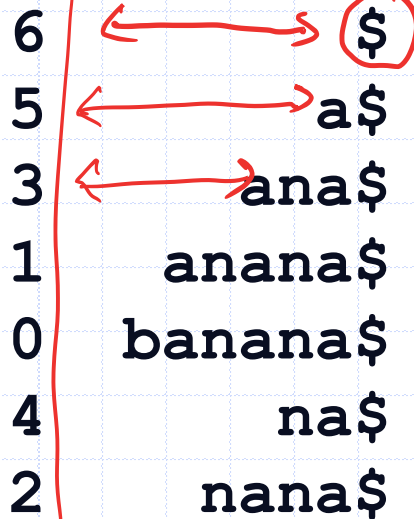
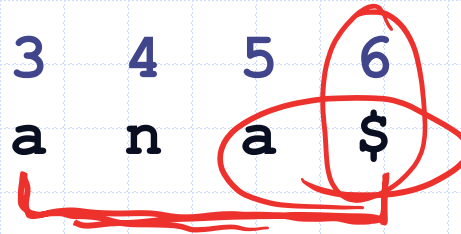


i = **0** **1** **2** **3** **4** **5** **6**
T = **b** **a** **n** **a** **n** **a** **\$**

Suffix Array Example

$i = 0 \quad 1 \quad 2$
 $T = b \quad a \quad n$

$3 \quad 4 \quad 5 \quad 6$
 $a \quad n \quad a \quad \$$



6 $\longleftrightarrow$ $\$$
5 $\longleftrightarrow$ $a\$$
3 $\longleftrightarrow$ $ana\$$
1 $\longleftrightarrow$ $anana\$$
0 $\longleftrightarrow$ $banana\$$
4 $\longleftrightarrow$ $na\$$
2 $\longleftrightarrow$ $nana\$$

Suffix Array Example

i = 0 1 2 3 4 5 6
T = b a n a n a \$

6 \$
5 a\$
3 ana\$
1 anana\$
0 banana\$
4 na\$
2 nana\$

SA = [6, 5, 3, 1, 0, 4, 2]

Suffix Array Example

i	=	0	1	2	3	4	5	6
T	=	b	a	n	a	n	a	\$

SA = [6, 5, 3, 1, 0, 4, 2]

Suffix Array Example

$i = 0 \quad 1 \quad 2 \quad 3 \quad 4 \quad 5 \quad 6$
 $T = b \quad a \quad n \quad a \quad n \quad a \quad \$$



$SA = [6, 5, 3, 1, 0, 4, 2]$

Does the string "bana" occur in T ?

Suffix Array Example

i	=	0	1	2	3	4	5	6
T	=	b	a	n	a	n	a	\$

SA = [6, 5, 3, 1, 0, 4, 2]

Does the string “bana” occur in T ?
Binary Search the SA!

Suffix Array Example

$i = 0 \quad 1 \quad 2 \quad 3 \quad 4 \quad 5 \quad 6$
 $T = b \quad a \quad n \quad a \quad n \quad a \quad \$$
 $SA = [6, 5, 3, 1, 0, 4, 2]$

Does the string "bana" occur in T ?
Binary Search the SA!

$a < \underline{b}ana$

$n > \underline{b}ana$

$b = b$

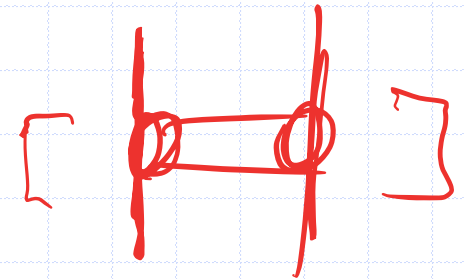
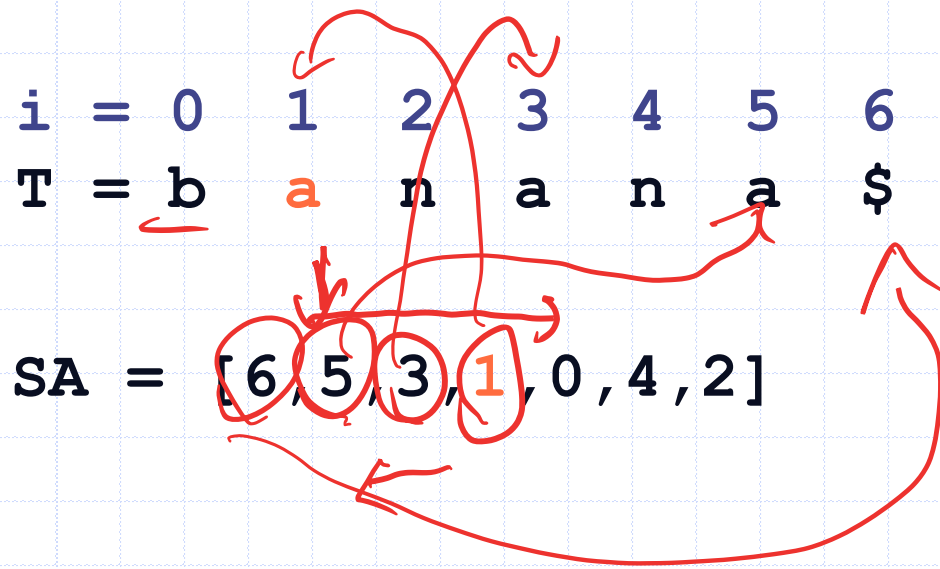
$ba = ba$

$ban = ban$

$bana = bana$

Yes

Suffix Array Example



ana

Does the string "bana" occur in T ?
Binary Search the SA!

Suffix Array Example

i	=	0	1	2	3	4	5	6
T	=	b	a	n	a	n	a	\$

'a' < 'b' –
go higher

SA = [6, 5, 3, 1, 0, 4, 2]

Does the string “bana” occur in T ?
Binary Search the SA!

Suffix Array Example

i	=	0	1	2	3	4	5	6
T	=	b	a	n	a	n	a	\$

SA = [6, 5, 3, 1, 0, 4, 2]

Does the string “bana” occur in T ?
Binary Search the SA!

Suffix Array Example

i	=	0	1	2	3	4	5	6
T	=	b	a	n	a	n	a	\$

SA = [6, 5, 3, 1, 0, 4, 2]

Does the string “bana” occur in T ?
Binary Search the SA!

Suffix Array Example

i	=	0	1	2	3	4	5	6
T	=	b	a	n	a	n	a	\$

$SA = [6, 5, 3, 1, 0, 4, 2]$

Does the string "bana" occur in T ?
Binary Search the SA!

Suffix Array Example

i	=	0	1	2	3	4	5	6
T	=	b	a	n	a	n	a	\$

'n' > 'b' –
go lower

SA = [6, 5, 3, 1, 0, 4, 2]

Does the string “bana” occur in T ?
Binary Search the SA!

Suffix Array Example

i	=	0	1	2	3	4	5	6
T	=	b	a	n	a	n	a	\$

SA = [6, 5, 3, 1, 0, 4, 2]

Does the string “bana” occur in T ?
Binary Search the SA!

Suffix Array Example

i	=	0	1	2	3	4	5	6
T	=	b	a	n	a	n	a	\$

$SA = [6, 5, 3, 1, 0, 4, 2]$

Does the string "bana" occur in T ?
Binary Search the SA!

Suffix Array Example

$i = 0 \quad 1 \quad 2 \quad 3 \quad 4 \quad 5 \quad 6$
 $T = \text{b a n a n a \$}$

$SA = [6, 5, 3, \cancel{1}, \underline{0}, 4, 2]$

Does the string “bana” occur in T ?
Binary Search the SA!

Suffix Array Example

i	=	0	1	2	3	4	5	6
T	=	b	a	n	a	n	a	\$

SA = [6, 5, 3, 1, 0, 4, 2]

Does the string “bana” occur in T ?

Yes, and it occurs at index 0.

Suffix Array Example

i	=	0	1	2	3	4	5	6
T	=	b	a	n	a	n	a	\$

SA = [6, 5, 3, 1, 0, 4, 2]

Note the trivial extension to support **count** queries.
“How many times does *bana* occur in *T*?”

Suffix Array Analysis

- Can be constructed in linear time, $O(n)$
Out of scope for this course
- Space occupancy is empirically better than the suffix tree
Just store T and a list of $|T|$ integers!
- May support a limited subset of suffix tree operations, but can be augmented to achieve more functionality

Further Reading

- Data Structures and Algorithms in Python
 - Chapter 13.1, 13.2, 13.5
- Introduction to Algorithms
 - Chapter 32